통합반 5회차 — 낭독 대본

[이론 01] 질문하는 AI와 일하는 AI의 차이 이해하기

[SLIDE 01] 정보와 도구로 일하는 AI 에이전트

안녕하세요. 이번 회차에서는 에이전트를 다룹니다. 지난 회차에서는 문서에서 뽑은 지식을 검토하고, 처음 정한 질문에 답할 수 있는지 확인했습니다. 이번에는 그 지식과 규칙을 도구로 읽고, 업무에 활용하는 AI를 살펴보려 합니다.

쿨러 성능 저하 경보가 났던 장면을 떠올려 보세요. 경보가 나고 운전원이 조치를 승인하기까지 2분이 채 안 걸렸습니다. 그런데 그 짧은 시간에 일을 대부분 한 건 사람이 아니었어요. 에이전트였죠. 그럼 에이전트는 그동안 뭘 읽고, 어떤 순서로 일했을까요? 그리고 근거가 모자랐다면, 언제 멈췄어야 했을까요?

이번 회차에서는 이 물음을 따라가 보겠습니다. 먼저 질문에 답하는 AI와 일하는 AI가 어떻게 다른지 가르고요. 다음으로 에이전트가 정보와 도구를 쓰는 원리를 본 뒤, 실습에서 에이전트를 직접 만들어 보겠습니다.

[SLIDE 02] 학습목표

그 길을 다 따라가고 나면 무엇을 할 수 있어야 하는지, 학습목표부터 짚어 볼게요. 화면의 카드 네 장이에요. 첫째 카드는 세 방식의 구분입니다. 같은 요청을 넣었을 때 LLM, RAG, 에이전트가 각각 무엇을 근거로 뭘 돌려주는지 견주어 봅니다. 둘째 카드는 실행 고리와 산출물이에요. 에이전트가 일한 순서를 실제 처리 기록에서 짚고, 그 끝에 나온 추천안이 단순한 답변과 어떻게 다른지 설명할 수 있으면 됩니다.

셋째 카드는 정보원과 멈춤 조건입니다. 에이전트가 읽는 정보원을 물음에 맞게 가르고, 근거가 모자랄 때 왜 멈춰야 하는지 까닭을 들 수 있어야 해요. 마지막 카드가 실습이고요. 에이전트를 직접 구성해서 질문해 보고, 그 결과를 근거와 대조해 봅니다.

[SLIDE 03] 목차

이 목표들을 어떤 순서로 다룰지는 목차에 있습니다. 이론 일곱 단원을 먼저 하고, 마지막 줄의 실습 넷으로 넘어가요.

위의 세 단원은 질문하는 AI와 일하는 AI의 차이를 다룹니다. LLM과 RAG, 에이전트를 구분하고요. 쿨러 사건의 처리 기록에서 에이전트의 실행 고리를 따라간 다음, 그 끝에 나온 업무 산출물이 단순한 답변과 뭐가 다른지 봅니다.

넷째부터 일곱째 단원까지는 에이전트가 정보와 도구를 쓰는 원리예요. 에이전트가 읽는 정보원과 도구, 지식과 현재값의 구분, 멈춰야 하는 조건, 그리고 결과와 과정을 남기는 구조화 출력과 실행

기록 순서로 갑니다. 실습에서는 에이전트 구성, 원인 추론, 접수 처리, 센서 탐색, 이 네 가지를 하나씩 해 보면서 이 원리를 직접 확인해 볼 거예요.

이번 시간에는 그 가운데 위의 세 단원, 질문하는 AI와 일하는 AI의 차이를 다룹니다.

[SLIDE 04] LLM · RAG · 에이전트의 구분

그럼 첫 단원, LLM과 RAG, 에이전트의 구분부터 시작하겠습니다. 쿨러 사건에서 에이전트가 맡았던 일을, 우리가 흔히 쓰는 채팅형 AI에게 그대로 맡긴다고 해 볼까요? 뭐가 모자랄까요? 경보 한 건의 처리를 맡기려면 AI에 무엇이 더 있어야 하는지, 세 방식을 같은 요청으로 견주면서 찾아보겠습니다.

[SLIDE 05] 경보에서 승인까지: 에이전트가 맡은 구간

먼저 쿨러 사건의 시간표에서, 에이전트가 맡았던 구간이 어디인지 볼게요. 그림은 왼쪽에서 오른쪽으로 읽으면 됩니다. 맨 왼쪽 종 모양이 쿨러 성능 저하 경보가 난 순간이고, 맨 오른쪽 사람 그림이 운전원이 승인한 순간이에요. 정확한 시각은 그림마다 붙은 이름표에 적혀 있어요. 둘 사이가 2분이 채 안 됩니다.

아래 시간 축으로 내려가 볼게요. 진하게 칠한 긴 구간이 1분 반쯤 됩니다. 화면에는 약 94초로 적었어요. 이 시간 동안 일한 건 사람이 아니라 가운데 에이전트입니다. 에이전트 그림 둘레에 원호가 감겨 있죠. 같은 일을 한 번이 아니라 여러 번 오가며 했다는 표시예요. 그 결과로 오른쪽의 겹친 카드, 추천안이 나왔습니다. 카드가 겹쳐 있는 건 추천 하나에 대안 셋이 함께 들어 있어서고요.

그 옆에 옅게 칠한 짧은 구간은 십여 초예요. 추천안이 나온 뒤 운전원이 승인하기까지 걸린 시간이죠. 그러니까 이 2분 가운데 대부분은, 에이전트가 판단을 준비하는 데 쓴 시간이었던 겁니다.

이 비율이 말해 주는 게 있어요. 사람이 한 일은 준비된 추천안을 보고 승인할지를 정하는 데 모였습니다. 원인을 찾고 후보를 견주는 긴 준비는 에이전트가 맡았고, 마지막 결정은 사람이 내린 거죠. 이번 회차에 들여다볼 건 바로 그 긴 준비 구간입니다.

[SLIDE 06] 94초 동안 이어진 네 가지 일

그 94초 동안 에이전트는 뭘 했을까요? 화면에 네 칸으로 놓았습니다. 값을 읽고, 원인을 가리고, 후보를 견주고, 추천안을 제출했어요.

첫 칸은 센서 값을 읽는 일이에요. 유온, 냉각 효율, 팬 진동, 캐비닛 온도를 읽었습니다. 경보는 유온이 오른다고 알렸는데, 왜 팬 진동과 캐비닛 온도까지 읽었을까요? 유온만 봐서는 원인을 가를 수가 없거든요. 팬이 제대로 도는지, 주변이 더운지를 따로 봐야 원인 후보를 하나씩 지울 수 있습니다. 둘째 칸에서는 그 값으로 원인을 가렸어요. 후보 셋 가운데 쿨러 핀 오염이 남았죠. 셋째 칸에서는 조치 후보 넷을 견주었습니다. 규정에 걸리는지, 조치하고 나면 유온이 얼마가 될지, 순위는 어떻게 되는지를 따졌고요. 마지막 칸에서 그 결과를 추천안으로 냈습니다.

이 사례에서는 뒤의 일이 앞의 결과를 필요로 합니다. 값을 읽어야 원인을 가릴 수 있고, 원인이 정해져야 어떤 조치를 견줄지가 정해지거든요. 견준 결과가 있어야 제출도 할 수 있고요.

결국 이 1분 반은 질문 하나에 답 하나를 만든 시간이 아닙니다. 앞 단계의 결과를 받아서 다음 단계를 하는, 그런 일이 줄줄이 이어진 시간이에요.

[SLIDE 07] LLM의 응답 생성과 외부 실행의 구분

이 일을 맡길 AI의 출발점은 LLM, 대규모 언어 모델입니다. 앞 회차에서 문서의 지식 후보를 뽑을 때도 썼죠. 우선 도구가 연결되지 않은 모델의 응답 한 번을 보겠습니다.

왼쪽 입력에는 질문과 지시, 참고 자료가 들어갑니다. 모델은 학습한 내용과 지금 받은 자료를 바탕으로 오른쪽 응답을 만들어요. 예를 들어 매뉴얼을 넣고 점검 조건을 정리해 달라고 하면, 문장을 읽고 조건을 정리할 수 있습니다.

그런데 지금 설비의 유온은 어떨까요? 입력에 없고 별도로 조회하지도 않았다면 모델은 현재 측정값을 확인할 수 없습니다. 답에 숫자를 적었다는 것과 센서 값을 읽었다는 것은 다른 일이에요. 화면에 외부 저장소로 가는 화살표가 없는 이유가 이것입니다.

도구를 연결한 채팅 서비스를 써 보셨다면 질문 한 번에 검색까지 해 주는 걸 보셨을 거예요. 그때는 모델만 있는 것이 아니라, 모델의 요청을 받아 검색을 실행하는 프로그램도 함께 동작합니다. 모델은 도구를 부르겠다는 요청을 만들 수 있고, 실제 조회는 연결된 실행 프로그램이 맡아요. 이 둘을 구분해야 뒤에서 에이전트가 한 일을 정확히 읽을 수 있습니다.

[SLIDE 08] LLM만으로는 채울 수 없는 세 자리

예를 하나 들어 볼게요. LLM에게 HYD-01의 유온이 지금 몇 도냐고 물으면 어떻게 될까요? 그 답은 시계열 DB에 있습니다. 모델이 학습한 내용 안에는 없죠. 이렇게 LLM만으로는 채울 수 없는 자리가 화면에 세 개 있어요.

첫째 카드는 지금의 값입니다. 유온, 냉각 효율, 운전 모드는 계속 바뀌어요. 학습이 끝난 뒤에 생긴 값이 모델 안에 있을 수는 없겠죠.

둘째 카드는 이 공장의 문서예요. 모델이 그 문서를 학습했는지, 최신 승인본의 조건까지 알고 있는지 우리는 보장할 수 없습니다. 일반적인 유압 설비 설명이 맞더라도 이 공장의 현재 기준과 같다는 뜻은 아니죠. 그래서 승인된 매뉴얼을 따로 제공하거나 찾아 읽게 해야 합니다.

셋째 카드가 수행이에요. 조회하고, 계산하고, 제출하는 일은 글을 만드는 일과 다릅니다. 모델이 「재고를 조회해 보겠습니다」라고 쓸 수는 있어요. 그렇다고 실제로 조회가 일어난 건 아니죠.

정리하면 값, 문서, 수행. 이 세 자리가 비어 있는 겁니다. 과정 처음에 AI에게 그대로 물으면 그럴듯한 답은 나오지만 그 근거를 확인할 방법이 없다고 했죠. 그 문제를 자리로 나눠 보면 바로 이 셋이에요. 지금의 값을 보지 못하고, 이 설비의 승인된 문서를 확인하지 못하고, 직접 확인해 볼 손발이 없는 겁니다.

[SLIDE 09] RAG의 근거 검색

이 가운데 문서의 빈자리를 메우는 방식이 RAG, 검색 증강 생성입니다. 모델이 배우지 않은 문서로도 답하게 해 주죠. 방법은 간단해요. 질문과 가까운 문서 조각을 먼저 찾고, 그 조각을 질문과 함께 모델에 넣는 겁니다.

화면의 네 칸을 왼쪽부터 볼게요. 먼저 사람이 질문을 합니다. 다음 칸이 검색이에요. 질문과 뜻이 가까운 문서 조각을 찾죠. 셋째 칸, 보강에서는 찾은 조각을 질문과 함께 모델에 넣습니다. 마지막 칸에서 모델이 그 조각을 읽고 글로 답하고요.

여기서 중요한 건 가운데 두 칸이에요. 모델을 다시 학습시키는 게 아닙니다. 답할 때 필요한 문서를 옆에 펼쳐 주는 거죠. 그래서 학습하지 않은 문서의 내용으로도 답할 수 있게 됩니다.

지난 회차에서 지식 그래프와 RAG를 함께 쓸 수도 있다고 했죠. RAG의 핵심은 필요한 자료를 검색해서 답의 근거로 제공하는 데 있습니다. 화면은 문서 조각을 찾는 예지만, 그래프나 데이터베이스에서 조회한 자료도 활용할 수 있어요. 문서 검색은 RAG의 한 구성이지, RAG가 다룰 수 있는 자료의 전부는 아닙니다.

이 그림의 마지막은 검색한 자료로 답을 만드는 단계입니다. 쿨러 경보의 처리를 맡기려면 여기서 더 정해야 할 것이 있어요. 어떤 값을 조회할지, 받은 결과에 따라 무엇을 더 확인할지, 언제 제출하거나 멈출지입니다. 에이전트는 이런 다음 행동을 정하는 구조이고, 그 안에서 자료를 찾을 때 RAG를 쓸 수도 있습니다.

[SLIDE 10] 에이전트의 목표 지향 작업 수행

그 수행의 빈자리까지 메우는 게 에이전트입니다. 조회하고, 계산하고, 제출하는 일까지 맡기려면 어떻게 해야 할까요? 모델이 도구를 부르고, 그 결과를 보면서 다음 행동을 정하게 해야 합니다.

맨 위 줄의 정의부터 볼게요. 이 수업의 에이전트는 목표를 받고, 도구의 결과를 보며 다음 행동을 정하는 시스템입니다. 필요한 일을 이어 가되, 완료 조건이나 멈춤 조건에 이르면 끝냅니다. 결과는 근거가 붙은 답변일 수도 있고, 다음 업무로 넘길 추천안일 수도 있어요.

예를 들어 추천안을 제출하라는 목표라면, 추천 문장을 썼다는 것만으로는 끝나지 않습니다. 필요한 근거를 확인했고, 정해진 항목을 채웠고, 제출 도구가 접수를 확인했는지를 봐야 해요. 반대로 필수 근거가 없거나 권한 밖의 일이 필요하면 완료하지 못한 이유를 남기고 멈춥니다. 스스로 끝났다고 말하는 것과 실제 완료 조건을 만족하는 것을 구분하는 겁니다.

가운데 줄이 도구예요. 모델이 부를 수 있도록 미리 정해 둔 기능입니다. 이름과 입력이 정해져 있어서, 모델은 어느 도구에 뭘 넣을지만 적으면 돼요. 이게 왜 중요할까요? 모델이 아무렇게나 적은 글은 시스템이 받아서 실행할 수가 없거든요. 실행 프로그램은 정해진 이름과 입력 형식, 허용된 권한을 확인해 그 요청을 실제 도구에 전달합니다. 쿨러 사건에서는 원인 진단, 생산오더 조회, 판단 제출 같은 도구가 쓰였죠.

맨 아래 줄이 쿨러 사건의 실제 에이전트입니다. 이름이 냉각 긴급 대응 에이전트예요. 경보 한 건을 받아서 원인을 가리고 추천안을 제출했습니다. 앞에서 본 1분 반 남짓이 바로 이 에이전트가 일한 시간이었고요.

[SLIDE 11] 연결한 기능이 다른 세 구성의 비교

이번에는 연결한 기능이 다른 세 구성을 같은 요청으로 비교해 볼게요. HYD-01의 쿨러 성능 저하 경보에 조치를 정하라는 요청입니다. 표의 오른쪽 열은 아직 보지 말고, 각 방식이 무엇을 돌려줄지 먼저 생각해 보세요.

첫 줄, LLM은 학습한 내용을 근거로 이어질 글을 만듭니다. 그러니 유압 설비에 대한 일반적인 설명이 돌아오죠. 쿨러 성능이 떨어지면 이런 걸 점검한다, 이런 식의 글이요.

둘째 줄, RAG는 찾아온 문서 조각을 근거로 답해요. 매뉴얼을 찾았다면 그 내용을 인용한 설명이 돌아옵니다. 이 공장의 문서를 반영하니까 한 걸음 나아간 거죠. 이 예에서는 문서만 제공했으므로 지금 유온은 확인하지 못합니다. 현재 값을 조회해 제공하는 구성이라면 이 부분은 달라질 수 있어요.

셋째 줄, 에이전트는 도구가 돌려준 결과를 근거로 삼아요. 지금의 값을 읽고, 원인을 가리고, 조치 후보 넷을 견주어서 제출합니다. 그래서 이 설비의 값으로 만든 추천안이 돌아오죠.

그럼 위의 두 줄은 쓸모가 없을까요? 그렇지는 않아요. 쿨러가 어떤 원리로 식히는지, 매뉴얼에 무슨 절차가 있는지 이해하는 데는 도움이 됩니다. 다만 운전원이 지금 이 경보에 승인 버튼을 누를지 정하는 재료로는 모자라요. 그러려면 이 설비의 지금 값과, 그 값에서 나온 근거가 붙은 결과가 있어야 하니까요.

이 표는 LLM, RAG, 에이전트를 서로 대체하는 세 세대로 나눈 것이 아닙니다. 모델은 응답을 만들고, RAG는 검색한 근거를 제공하고, 에이전트는 결과를 보며 다음 행동을 정합니다. 함께 쓰일 수 있는 요소들이죠. 우리 업무에 무엇이 필요한지는 답의 길이가 아니라, 확인할 근거와 수행할 일로 정하면 됩니다.

[SLIDE 12] 에이전트의 세 요소: 모델 · 도구 · 실행 고리

그렇게 찾고 계산하고 제출하는 에이전트는, 무엇으로 이루어져 있을까요? 이 수업에서 보는 LLM 기반 에이전트는 크게 요소가 셋이에요. 모델, 도구, 실행 고리입니다. 여기서 눈여겨볼 게 있어요. 모델은 앞에서 본 LLM 그대로라는 겁니다.

화면 왼쪽 말풍선이 모델이에요. 글을 읽고 다음 행동을 정합니다. 가운데가 도구고요. 지식 그래프와 시계열 DB, 표를 실제로 조회하고 계산해서 결과를 돌려주죠. 그다음 원호와 스패너가 실행 고리입니다. 모델과 도구 사이를, 도구 결과에 따라 다음 호출이나 종료를 정하도록 이어 주는 틀이에요.

이 셋을 더한 게 등호 오른쪽의 에이전트입니다. 모델이 더 똑똑해진 게 아니에요. 모델 둘레에 도구와 고리가 붙은 거죠.

그래서 에이전트가 일을 잘하는지 따질 때 모델만 봐서는 안 됩니다. 어떤 도구를 쓸 수 있는지, 고리가 어떻게 돌아가는지가 결과를 함께 정하거든요.

[SLIDE 13] 에이전트를 정의하는 항목: 목표 · 도구 · 업무 지침

요소는 셋으로 같은데, 에이전트 하나하나는 무엇으로 정해질까요? 화면에 두 에이전트의 정의를 나란히 놓았습니다. 왼쪽은 쿨러 사건을 맡은 냉각 긴급 대응 에이전트, 오른쪽은 예비품 재고 경보를 맡는 구매 에이전트예요.

첫 줄은 목표입니다. 무엇을 이뤄야 일이 끝나는지를 정하죠. 왼쪽은 쿨러 경보의 원인을 진단하고, 냉각 조치 후보를 견주어 추천안을 올리는 게 목표예요. 오른쪽은 재고가 기준을 벗어났을 때 필요한 양을 정하고, 공급사를 금액과 납기와 품질로 견주어 발주 추천안을 올립니다.

목표 칸의 화살표 끝을 견주어 보세요. 둘 다 마지막에 올릴 것으로 끝나죠. 왼쪽은 추천안, 오른쪽은 발주 추천안이요. 목표에 마지막에 무엇을 내야 하는지가 들어 있어야, 에이전트가 언제 일이 끝났는지를 알 수 있습니다.

둘째 줄은 업무에서 쓰는 도구예요. 왼쪽은 설비의 지식과 현재 상태, 조치 평가가 필요하고, 오른쪽은 재고와 공급사의 조건을 비교하는 조회가 필요합니다. 두 에이전트가 같은 서버의 도구를 공유하더라도 실제 업무에서 필요한 기능은 다를 수 있어요. 이 표는 그 쓰임을 비교한 것입니다. 개별 호출의 허용 여부는 등록된 권한 목록을 따로 확인해야 해요.

셋째 줄은 업무 지침입니다. 어떤 순서로 일할지를 적어 둔 거고, 에이전트 스킬이라는 파일로 줘요. 파일이 어떻게 생겼는지는 따로 다룰 테니까, 지금은 순서를 정해 주는 지침이라는 것만 기억해 두면 됩니다.

목표와 도구와 지침이 다르면, 같은 모델이라도 전혀 다른 일을 하는 에이전트가 돼요. 실습에서 에이전트를 직접 만들 때도 바로 이 세 가지를 정하게 됩니다.

[SLIDE 14] 도구 호출 한 건: 모델이 적는 것과 도구가 돌려주는 것

에이전트가 도구를 부른다는 게 실제로는 어떤 모습일까요? 쿨러 사건의 처리 기록에서 도구 호출 한 건을 꺼내 왔습니다. 경보가 나고 10초쯤 뒤에 원인 진단 도구를 부른 장면이에요.

화면 왼쪽 맨 윗줄에 도구 이름이 있죠. 원인 진단, 하는 일이 정해진 기능입니다. 그 아래 물음은 모델이 적은 입력이에요. 어느 설비인지, 어떤 종류의 경보인지, 경보 번호가 뭔지, 이 세 가지를 넣었습니다.

물음에 든 게 이 세 값뿐이라는 점도 보세요. 지식 그래프에서 무엇을 찾을지, 어떤 조회문으로 값을 읽을지는 모델이 적지 않았습니다. 그건 도구 쪽에 이미 정해져 있어요. 모델이 할 일은 무엇에 대해 물을지를 정확히 적는 것까지인 거죠.

맨 아래 받음은 도구가 돌려준 결과예요. 1만 자가 넘는 긴 JSON입니다. 이 결과를 만들려고 지식 그래프를 읽고 시계열 DB를 조회한 건 모델이 아니라 도구고요. 맨 윗줄 괄호에는 이 호출 한 번에 걸린 시간이 적혀 있어요. 0.2초 정도입니다.

모델이 만든 것은 도구 이름과 입력을 담은 호출 요청입니다. 실행 프로그램이 그 요청을 도구에 전달하고, 도구는 조회 결과를 돌려줍니다. 그러니 진행 설명에 조회하겠다고 적힌 것만 보지 말고, 실제 호출과 돌아온 결과까지 봐야 수행 여부를 알 수 있어요.

이쯤 되면 단원 처음의 물음에 답할 수 있어요. 경보 한 건을 맡기려면 AI에 무엇이 더 있어야 할까요? 모델은 그대로 둡니다. 거기에 모델이 적은 요청을 받아 실제로 조회하고 계산할 도구, 그리고 그 일을 목표까지 이어 갈 틀을 붙이는 거예요.

[SLIDE 15] 에이전트의 실행 고리

방금 모델 둘레에 붙이는 것 가운데, 일을 목표까지 이어 가는 틀이 있었죠. 그 틀을 들여다보는 게 두 번째 단원, 에이전트의 실행 고리입니다. 경보를 받은 에이전트는 그 1분 반 동안 어떤 순서로 일했을까요? 그리고 왜 도구를 한 번 부르는 걸로 끝나지 않고, 고리처럼 이어져야 했을까요? 쿨러 사건의 처리 기록을 요청 해석부터 종료까지 따라가면서 답을 찾아보겠습니다.

[SLIDE 16] 결과를 확인한 뒤 다음 입력을 정하는 까닭

원인 진단을 부르면 결과를 받을 수 있습니다. 그런데 조치 후보를 평가하려면, 그 결과에 나온 원인을 다음 도구의 입력으로 넣어야 해요. 아직 원인을 읽지 않았다면 다음 입력도 정할 수 없겠죠. 중요한 것은 사용자에게 답을 몇 번 보여 주느냐가 아니라, 필요한 결과를 실제로 확인했느냐입니다.

화면 왼쪽은 값을 확인하지 않고 결론부터 만드는 경우예요. 문장은 매끄러울 수 있지만, 어떤 원인을 근거로 조치를 골랐는지 확인할 수 없습니다. 오른쪽은 진단 결과를 받아 원인을 확인하고, 그 원인에 맞는 후보를 평가합니다. 중간에 필요한 결과를 얻지 못하면 보완하거나 멈춰야 하고요.

이런 순서를 모두 에이전트가 정해야 하는 것은 아닙니다. 진단 다음에 평가를 부르는 절차가 늘 같다면, 프로그램에 그 순서를 고정해 둘 수도 있어요. 이것을 워크플로라고 부릅니다. 에이전트는 결과를 보고 다음에 어떤 기능을 쓸지 정할 필요가 있을 때 도움이 됩니다. 정해진 흐름 안의 일부 일만 에이전트에게 맡길 수도 있죠.

그래서 모든 업무를 에이전트로 바꿀 필요는 없습니다. 고정된 절차로 충분한 부분은 그대로 두고, 상황에 따라 선택해야 하는 부분에 판단의 여지를 주면 돼요. 쿨러 사례에서는 업무 지침이 기본 순서를 주고, 에이전트가 호출 결과를 확인하며 일을 이어 갑니다. 그 구조를 다음 그림에서 펼쳐 보겠습니다.

[SLIDE 17] 실행 고리: 요청 해석에서 종료까지

화면의 다섯 원은 그 실행 고리를 단계로 펼친 거예요. 에이전트를 설명할 때 널리 쓰이는 틀이고, 영어로는 Agent Loop라고 부릅니다. 원마다 그림이 하나씩 들어 있어요. 문서, 목록, 스패너, 돋보기, 그리고 확인 표시가 붙은 문서. 그림을 단계 이름과 짝지어 보면서 왼쪽부터 하나씩 볼게요.

첫째는 요청 해석입니다. 무엇을 받았고 무엇을 내야 하는지 파악하는 단계예요. 둘째는 계획이고요. 목표를 이루려면 어떤 순서로 할지 정합니다. 셋째는 도구 선택과 호출이에요. 어느 도구에 무엇을 물을지 정해서 부르죠. 넷째는 결과 관찰입니다. 받은 결과를 보고 목표에 이르렀는지 판단해요.

목표에 이르렀으면 마지막 종료로 갑니다. 산출물을 제출하고 끝내는 단계죠. 그런데 아직 못 미쳤거나 호출이 뜻대로 안 됐다면 어떻게 할까요? 계속 진행할 근거와 권한이 있다면 위쪽 점선을 따라 계획으로 되돌아갑니다. 계획을 고쳐 다음 호출을 하거나 다시 시도합니다. 필수 근거가 없거나 정한 한도를 넘었다면, 목표를 이루지 못했더라도 이유를 남기고 멈춰야 해요.

다섯 단계 가운데 고리를 고리답게 만드는 건 넷째, 돋보기가 그려진 결과 관찰이에요. 관찰이 없다면 에이전트는 처음 세운 계획을 끝까지 그냥 밀고 가겠죠. 결과가 비었든 틀렸든 상관없이요. 관찰이 있으니까 자기가 한 일의 결과를 보고 다음을 고칠 수 있는 겁니다.

쿨러 사건의 에이전트도 이 다섯 단계를 그대로 밟았어요. 원마다 붙은 이름표의 둘째 줄이 쿨러 사건에서 그 단계에 해당하는 것입니다. 요청 해석은 업무와 입력과 결과 형식을 받는 일이고, 계획은 지침의 다섯 단계, 호출은 원인 진단과 대안 평가, 관찰은 원인 1순위와 결과의 크기, 종료는 판단 제출과 결과 파일이죠.

[SLIDE 18] 실제 기록 ①: 요청 해석

고리의 첫 단계, 요청 해석부터 실제 기록에서 확인해 보죠. 화면 왼쪽은 처리 기록의 앞부분입니다. 첫 줄에서 경보가 나고 처리 건이 열렸어요. 그리고 1초 뒤, 둘째 줄에서 에이전트 단계가 시작됩니다. 둘째 줄은 이 에이전트에게 배정된 스킬이 하나라는 기록이에요. 앞에서 에이전트를 정의하는 항목에 업무 지침이 있었죠. 그 지침이 실제로 여기서 일에 붙은 겁니다.

에이전트는 시작하면서 몇 가지를 받아요. 오른쪽 첫 설명이 그거예요. 맡은 업무와 지시사항, 입력, 작업 공간, 그리고 결과를 낼 형식입니다. 입력에는 설비 이름과 경보 번호가 들어 있고요. 작업 공간은 이 실행에만 쓰는 전용 폴더예요. 에이전트가 만드는 파일은 이 안에만 쓰고, 끝에 결과 파일을 남기는 곳도 여기입니다. 결과 형식은 무엇을 어떤 항목으로 내야 하는지를 정한 거죠.

결과 형식을 처음부터 받는다는 게 중요해요. 끝에서 무엇을 내야 하는지 알아야, 거기에 이르는 길을 거꾸로 세울 수 있거든요. 원인을 적어 내야 한다면 원인을 가리는 일이 필요하고, 추천과 대안을 내야 한다면 후보를 견주는 일이 필요하겠죠.

기록을 보면, 에이전트가 가장 먼저 한 일은 업무 지침을 읽는 거였어요. 셋째 줄의 스킬 읽음이 그 장면입니다. 냉각 긴급 대응용 에이전트 스킬을 읽고, 무엇을 어떤 순서로 할지 파악한 거죠.

맨 아래 줄은 에이전트가 직접 적은 진행 설명이에요. 먼저 스킬 절차를 읽고, 필요한 판단 도구를 불러오겠다고 적었습니다. 에이전트는 호출 사이사이에 이렇게, 지금 뭘 하려는지를 말로 남겨요.

이 진행 설명은 처리 기록에 그대로 남습니다. 나중에 사람이 기록을 읽을 때, 에이전트가 왜 그 도구를 불렀는지 알 수 있는 단서가 되죠.

진행 설명은 기록을 읽는 데 도움을 주지만, 모델 내부의 생각을 그대로 보여 주거나 실행 성공을 증명하는 것은 아닙니다. 실제로 지침을 읽었는지, 도구가 결과를 돌려줬는지는 해당 호출 기록과 함께 확인합니다.

[SLIDE 19] 계획: 목표를 도구 호출의 순서로 푼 것

고리의 둘째 단계는 계획입니다. 원인을 가리고 조치를 추천하라, 이 목표는 그대로는 실행할 수가 없죠. 어느 도구를 어떤 순서로 부를지로 풀어야 합니다. 그렇게 풀어 놓은 게 계획이에요.

쿨러 사건에서는 업무 지침에 다섯 단계가 적혀 있었습니다. 화면의 다섯 칸이에요. 첫째, 입력에서 설비와 경보의 종류, 경보 번호를 읽습니다. 둘째, 원인 진단을 불러서 원인 1순위와 고장 유형을 얻고요. 셋째, 업무 DB에서 납기 오더와 출하 대기 로트를 확인해요. 넷째, 대안 평가로 후보마다 규정 판정과 예측 유온, 순위를 봅니다. 다섯째, 판단을 제출하고 판단 번호를 받죠.

에이전트는 이 다섯 단계를 뼈대로 삼았습니다.

화면에서 넷째 칸, 대안 평가를 강조했죠. 규정 판정, 예측 유온, 순위. 어디서 본 말들이죠? 조치 절차와 판단 규칙을 다룰 때 결정표로 제외와 감점을 가렸고, 조치하면 유온이 어떻게 될지 미리 계산했고, 성과 지표로 순위를 매겼습니다. 에이전트는 그 판단을 새로 해내는 게 아니에요. 이미 만들어 둔 규칙과 계산을 도구로 불러서 결과를 받는 겁니다.

그런데 하나 기억해 둘 게 있어요. 지침은 어디까지나 뼈대입니다. 실제로 어떤 값을 넣을지, 돌아온 결과를 어떻게 읽을지는 그때그때 에이전트가 정해요.

[SLIDE 20] 실제 기록 ②: 관찰한 결과가 다음 호출의 입력이 된다

돌아온 결과를 읽고 다음에 넣을 값을 정하는 모습, 이건 고리의 셋째와 넷째 단계인 도구 호출과 결과 관찰에서 보입니다. 화면 위쪽은 원인 진단이 돌려준 결과의 요지예요. 긴 결과 가운데 이 장에 필요한 두 줄만 옮겨 왔습니다. 아래쪽은 5초 뒤에 부른 대안 평가의 물음이고요.

위쪽 결과에서 에이전트가 읽은 건 두 가지입니다. 원인 1순위는 쿨러 핀 오염, 고장 유형은 쿨러 냉각 성능 상실이에요. 둘 다 지식 그래프의 식별자로 적혀 있죠. 오른쪽 마지막 설명처럼, 앞머리를 보면 원인인지 고장 유형인지가 나뉩니다.

이제 아래쪽 물음을 보세요. 위쪽 원인 1순위 줄의 값을 눈으로 따라 내려가면, 아래 물음의 원인 칸에 똑같이 들어가 있죠? 고장 유형도 마찬가지예요. 에이전트가 원인을 자기 말로 바꿔 적지 않고, 받은 값을 그대로 넘긴 겁니다. 진단에서 관찰한 결과가 다음 호출의 입력이 된 거예요. 원인을 모르면 후보를 평가할 수 없다고 했죠. 바로 그 연결이 여기서 보이는 거예요.

이 두 호출을 이어 주는 건 모델의 기억이 아니에요. 앞 호출이 돌려준 값입니다. 그래서 대안 평가가 어떤 원인을 바탕으로 했는지가 기록에 그대로 남아요.

그런데 비어 보이는 게 하나 있죠. 앞의 계획에서 셋째 단계는 업무 DB에서 납기 오더와 출하 대기 로트를 확인하는 거였는데, 이 기록에는 진단 다음에 바로 대안 평가가 옵니다. 오른쪽 넷째 설명이 그 답이에요. 이 실행은 업무 값 조회를 따로 부르지 않았습니다. 대안 평가 도구가 평가에 필요한 납기 같은 업무 값을 평가 안에서 직접 읽어 넣었거든요. 단계가 빠진 게 아니라, 다른 도구 안에서 채워진 거죠.

[SLIDE 21] 실제 기록 ③: 큰 결과를 읽는 방법을 바꾸려는 시도

고리가 늘 이렇게 한 번에 이어지지는 않습니다. 이번에는 받은 결과 때문에 방법을 바꾸겠다고 한 장면이에요. 대안 평가가 돌려준 결과가 너무 길었거든요. 12만 자가 넘었습니다. 앞에서 본 원인 진단의 결과보다 열 배 가까이 긴 거죠.

왼쪽 칸을 보세요. 에이전트가 도구의 결과를 한 번에 받을 수 있는 크기에는 한도가 있어요. 이 결과는 그 한도를 넘어서, 본문이 파일로 따로 빠졌습니다. 에이전트가 그 본문을 그대로 받지 못한 거죠. 에이전트는 이 상황을 보고 계획을 바꿉니다. 가운데 칸에서 순위와 규정 판정만 간추려 보겠다고 적었죠. 그리고 6초 뒤, 오른쪽 칸에서는 필요한 항목만 좁혀서 다시 뽑겠다고 했습니다. 따옴표 안은 에이전트가 기록에 남긴 말 그대로예요.

왜 하필 순위와 규정 판정이었을까요? 추천안에 들어갈 게 바로 그거거든요. 어느 후보가 몇 순위인지, 규정에 걸려 빠지거나 감점된 후보는 없는지. 받지 못한 결과를 통째로 다시 달라고 한 게 아니라, 목표에서 거꾸로 따져 꼭 필요한 것만 추린 겁니다.

이 화면으로 확인할 수 있는 것은 결과가 너무 컸다는 상황과, 필요한 항목을 좁혀 보겠다는 진행 설명입니다. 실제로 무엇을 다시 읽었고 필요한 내용을 모두 얻었는지는 이어지는 호출과 결과를 더 봐야 해요. 특히 짧게 만들려고 조치 제한이나 경고를 빼면 안 됩니다. 목표에 필요한 근거를 보존하면서 다룰 수 있는 크기로 읽어야 하는 거죠.

[SLIDE 22] 실제 기록 ④: 종료

고리의 마지막은 종료입니다. 에이전트 단계는 답 문장이 아니라, 추천안을 제출하는 걸로 끝났어요.

맨 윗줄이 판단 제출입니다. 추천안을 제출하는 도구를 불렀고, 판단 번호를 돌려받았죠. 시작한 지 45초쯤 됐을 때예요. 이 번호는 뒤에 만들어지는 작업지시에도 그대로 기록됩니다. 어떤 판단에서 나온 작업인지를 거슬러 찾을 수 있게 하는 거예요. 나중에 정비 기록을 보다가 이 작업이 왜 생겼는지 궁금해지면, 번호 하나로 어느 경보에서 어떤 근거로 나온 판단인지까지 따라갈 수 있습니다.

그다음 에이전트는 제출이 끝났다고 적고, 정해진 형식의 결과 파일을 썼습니다. 처음에 받은 작업 공간에 남기는 파일이죠. 맨 아래 두 줄은 기록의 원문이 아니라, 구간을 적어 둔 주석이에요. 여기서 에이전트 단계가 끝났고, 시작부터 1분 반 남짓 걸렸습니다.

제출 도구의 응답을 받은 시점과 에이전트 단계가 끝난 시점 사이에는 간격이 있습니다. 다만 이 기록만으로 그 시간을 전부 파일 작성에 썼다고 말할 수는 없어요. 이 장에서 확인할 것은 제출 접수와 최종 결과 저장이 서로 다른 완료 항목이라는 점입니다.

이렇게 놓고 보면 그 1분 반 안의 순서가 다 보이죠. 업무 지침을 읽어 요청을 해석하고, 다섯 단계로 계획을 세우고, 진단과 평가를 부르고, 큰 결과를 만났을 때 읽는 방법을 바꾸겠다고 설명하고, 제출과 결과 저장으로 끝냈습니다.

[SLIDE 23] 다른 실행의 기록: 같은 고리, 다른 길

그럼 같은 경보를 다른 실행에서 처리했을 때도 똑같이 일했을까요? 화면은 같은 쿨러 경보를 다른 실행에서 처리한 기록이에요. 이번에는 도구를 부른 순서를 처음부터 끝까지 놓았습니다. 같은 지침을 받았는데도, 앞의 기록과 다른 데가 있어요.

첫 줄은 스킬 파일을 읽는 요청 해석이에요. 둘째 줄부터 볼게요. 에이전트는 일을 시작하자마자 쓸 도구의 이름을 불러왔습니다. 연결된 도구는 더 많았지만, 이번 일에 필요한 것만 골랐어요. 실행 고리의 셋째 단계 가운데 도구 선택이, 이 기록에는 한 줄로 따로 남아 있는 거죠.

넷째 줄에서는 앞에서와 같은 일이 생겼습니다. 대안 평가의 결과가 너무 커서 받지 못했어요. 그런데 이 실행은 다른 길을 택했습니다. 다섯째 줄에서 생산오더와 출하 대기 로트를 읽은 다음, 여섯째 줄에서 조치 효과 예측을 세 번 불렀어요. 팬만 올렸을 때, 팬을 올리고 부하를 낮췄을 때, 부하만 낮췄을 때 15분 뒤 유온이 어떻게 될지를 하나씩 물은 거죠. 이건 지침의 다섯 단계에는 없던 호출입니다. 평가 결과를 못 받았으니까, 대안의 예상 유온을 다른 도구로 보완해 확인한 거예요. 순서도 하나 달라졌어요. 지침에서는 셋째였던 업무 값 확인이, 이 기록에서는 대안 평가 뒤인 다섯째 줄에 와 있죠.

마지막 메시지도 눈여겨볼 만합니다. 에이전트는 평가의 상세 결과를 열어 보지 못했다는 사실, 그리고 대신 무엇을 근거로 이유를 적었는지를 함께 남겼어요. 앞에서 지침은 뼈대라고 했죠. 어떤 도구를 어떤 차례로 부를지는 결과를 보며 에이전트가 정한다는 것, 이 기록이 바로 그걸 보여 줍니다.

이제 이 단원의 물음에 답할 수 있어요. 도구가 있는데 왜 한 번 부르고 끝내지 못할까요? 앞 호출의 결과를 보고서야 다음 호출의 입력을 적을 수 있고, 결과가 뜻대로 오지 않으면 방법을 바꿔야 하니까요. 그래서 일은 고리로 이어질 수밖에 없고, 같은 지침을 받아도 실행마다 고리를 도는 길이 달라질 수 있는 겁니다.

다만 다른 길을 택했다는 사실이 그 길의 타당성까지 보장하지는 않습니다. 유온 예측은 조치 효과를 비교하는 근거예요. 그 결과만으로 규정 위반이나 작업 제한까지 확인했다고 할 수는 없죠.

필수 판정이 빠졌다면 원래 결과를 확인하거나 보류해야 합니다. 실제 실행 기록은 잘된 점과 부족한 점을 함께 점검하는 자료로 읽어야 해요.

[SLIDE 24] 단순 답변과 업무 산출물

고리의 끝에서 에이전트가 낸 건 답 문장이 아니라 추천안이었죠. 그 차이를 다루는 게 세 번째 단원, 단순 답변과 업무 산출물입니다. 같은 에이전트라도 질문을 받으면 답을 냅니다. 답과 추천안은 뭐가 다를까요? 그리고 추천안은 제출된 뒤에 어디로 넘어갈까요? 이 물음에 답하면서, 질문하는 AI와 일하는 AI의 차이를 마무리해 보겠습니다.

[SLIDE 25] 답변과 업무 산출물의 차이

이 포털에서 에이전트를 쓰는 두 가지 경우를 비교해 보겠습니다. 화면 왼쪽이 단순 답변, 오른쪽이 업무 산출물이에요. 받는 쪽, 형식, 다음 행동, 이 세 줄로 견주어 보겠습니다.

왼쪽부터요. 단순 답변을 받는 쪽은 질문한 사람입니다. 형식은 답과 근거예요. 답할 수 없을 때는 답할 수 없다는 사실과 그 이유고요. 이 질문 기능은 답을 돌려주는 데서 처리가 끝납니다. 사람이 그 답으로 후속 일을 할 수는 있지만, 이 기능이 설비 조치까지 자동으로 이어 주지는 않아요.

오른쪽 업무 산출물은 받는 쪽부터 달라요. 승인할 담당자, 그리고 다음 단계의 시스템이 받습니다. 형식은 원인, 추천, 뒤진 대안, 설명, 판단 번호처럼 항목이 정해져 있고요. 다음 행동도 있습니다. 승인이 나면 설비 명령과 작업지시로 이어지죠.

업무 산출물은 왜 항목이 정해져 있어야 할까요? 사람만 읽는 게 아니라, 시스템이 받아서 다음 일을 하거든요. 항목이 빠지거나 모양이 다르면, 다음 단계가 그 결과를 쓸 수가 없습니다.

[SLIDE 26] 질문에 답하는 에이전트: 답과 근거

두 쪽을 실제 예로 하나씩 볼게요. 먼저 왼쪽, 단순 답변입니다. 에이전트에게 완제품 재고가 가장 적은 설비와 그 수량을 물었어요.

화면의 답을 보면, 2호기 설비이고 완제품 재고가 100개라는 내용입니다. 그런데 이 답은 에이전트가 기억으로 말한 게 아니에요. 업무 DB를 조회해서 얻은 값입니다.

에이전트는 도구를 두 번 불렀어요. 처음에는 어떤 표가 있는지를 확인했고, 그다음에 조회문을 실행했습니다. 이 순서에도 까닭이 있어요. 에이전트에게는 표와 열의 이름을 짐작해서 조회문을 쓰지 말고, 먼저 실제로 있는 표를 확인하라는 규칙이 있거든요. 그래서 확인 한 번, 조회 한 번이 된 겁니다.

답 아래 붙은 게 그 조회문이에요. 완제품 재고 표에서 수량이 적은 순으로 정렬해 읽었죠. 돌아온 몇 행을 적은 순으로 놓았으니, 맨 윗줄이 답이 된 거고요.

답할 수 없을 때도 형식이 정해져 있습니다. 지어내지 않고, 답할 수 없다는 사실과 그 이유를 내요. 답이 한 번으로 끝나더라도 에이전트는 도구로 읽은 값으로 답하고, 어디서 읽었는지를 근거로 남기는 겁니다.

[SLIDE 27] 쿨러 사건의 업무 산출물: 추천안에 담긴 것

이번엔 오른쪽, 업무 산출물입니다. 쿨러 사건의 추천안을 펼쳐 볼게요. 표의 네 줄이 추천안에 들어 있는 네 가지예요.

첫 줄은 원인입니다. 쿨러 핀 오염이고, 고장 유형은 쿨러 냉각 성능 상실이에요. 둘째 줄이 추천이고요. 팬 최대 운전이 1순위였습니다.

셋째 줄은 뒤진 대안 셋이에요. 부하를 낮추는 두 안이 같은 점수로 공동 2순위였습니다. 하나는 부하 70 %에 야간 핀 세척, 다른 하나는 팬 최대 운전에 부하 80 %예요. 원인을 없애는 쿨러 핀 세척은 4순위였고요. 이 점수와 순위는 조치 후보를 견줄 때 나온 결과 그대로입니다.

여기서 고개를 갸웃할 수 있어요. 원인을 없애는 조치가 왜 꼴찌일까요? 핀 세척은 작업지시를 내서 사람이 해야 하는 일이라, 지금 오르고 있는 유온을 당장 내리지는 못하거든요. 그래서 당장 유온을 잡는 조치들보다 뒤에 놓인 겁니다.

넷째 줄은 설명과 판단 번호예요. 추천이 왜 앞서고 대안이 왜 뒤지는지를 설명하고, 이 판단을 가리키는 번호를 붙였습니다.

여기서 중요한 건, 대안이 왜 뒤졌는지까지 함께 낸다는 점이에요. 담당자는 추천 하나만 보고 고르는 게 아닙니다. 무엇과 견주어서 왜 이 안이 나왔는지를 보고 결정할 수 있죠.

[SLIDE 28] 추천안이 넘어가는 곳: 승인 뒤의 명령과 작업지시

이 추천안이 제출된 뒤에 어디로 넘어가는지, 시각과 함께 놓았습니다. 다섯 칸이 에이전트, 사람, 시스템, 세 영역으로 묶여 있어요.

에이전트의 일은 맨 왼쪽 칸에서 끝납니다. 추천안을 내고 단계를 끝냈죠. 다음 칸이 사람의 영역이에요. 잠시 뒤 운전원이 추천안을 승인했습니다.

그 뒤는 전부 시스템이 합니다. 승인 1초 뒤에 팬 속도를 최대로 올리는 설비 명령이 나갔어요. 그 다음 재관측에서 유온이 내려가고 경보가 해제된 걸 확인했고요. 마지막으로 1분쯤 뒤에 핀 세척 작업지시가 등록됐습니다.

명령을 보낸 뒤에도 시스템의 일은 이어집니다. 재관측으로 조치가 정말 듣는지를 확인하죠. 그리고 유온이 내려갔어도 핀은 여전히 오염된 채예요. 경보가 꺼진 것과 원인이 없어진 건 다르니까, 마지막에 핀 세척 작업지시가 붙는 겁니다. 앞에서 4순위였던 세척이 버려진 게 아니라, 승인 뒤의 작업으로 이어진 거죠.

이 쿨러 업무에서 에이전트가 맡은 범위는 조사와 평가, 추천안 제출까지입니다. 추천안은 저장하지만 설비 제어 명령을 직접 내지는 않아요. 운전원이 승인한 뒤 시스템 단계가 명령과 작업지시를 이어 갑니다. 모든 에이전트의 공통 제약이 아니라, 이 업무에서 정해 둔 역할과 권한의 경계입니다.

앞에서 본 두 기능은 모두 에이전트를 활용합니다. 하나는 확인한 답을 사람에게 돌려주고, 다른 하나는 정해진 형식의 제안을 승인 절차로 넘겨요. 어떤 결과를 어디에 넘겨야 하는지 정해야, 에이전트의 완료 조건도 분명해집니다.

다음 시간에는 에이전트가 이 일을 하면서 정보와 도구를 어떻게 쓰는지, 그 원리를 봅니다.

[이론 02] 에이전트가 다양한 정보와 도구를 사용하는 원리 이해하기

[SLIDE 29] 에이전트의 정보원과 도구

이번 시간에는 에이전트가 정보와 도구를 쓰는 원리를 봅니다. 앞 시간에는 질문하는 AI와 일하는 AI가 어떻게 다른지 봤죠. 이번에는 에이전트가 그 일을 하면서 무엇을 어디서 읽는지, 언제 멈추는지, 결과와 과정을 어떻게 남기는지로 들어가요. 네 번째 단원은 에이전트의 정보원과 도구입니다. 쿨러 사건의 추천안에는 원인 후보, 측정값, 납기, 작업 절차가 들어 있었어요. 이 정보들은 각각 어디에 있었을까요? 그리고 에이전트는 어떤 길로 그걸 읽었을까요? 정보의 출처와 읽는 길을 차례로 따라가 보겠습니다.

이 원리를 확인한 뒤에는 이어질 실습 네 개에서 무엇을 만들고 어떤 결과를 확인할지도 함께 살펴보겠습니다.

[SLIDE 30] 추천안 한 건에 들어간 정보의 출처

팬 최대 운전, 이 추천안 한 건을 만드는 데 쓰인 정보를 출처별로 놓았습니다. 가운데가 에이전트고, 네 귀퉁이가 서로 다른 역할의 정보예요. 저장소가 반드시 네 개라는 뜻은 아닙니다. HYD에서는 온톨로지와 SOP를 같은 지식 그래프에서 읽어요.

왼쪽 위, 원과 선이 이어진 그림이 온톨로지입니다. 원인 후보 셋이 여기서 왔어요. 오른쪽 위는 시계열 DB고요. 냉각 효율의 측정값이 여기서 왔죠. 오른쪽 아래는 업무 DB예요. 납기까지 남은 시간이 여기 있었습니다. 왼쪽 아래는 SOP예요. 팬 최대 운전의 단계와 그 출처가 여기서 왔습니다.

정보원에서 에이전트로 가는 선을 보면, 선마다 작은 스패너가 있죠. 도구를 뜻합니다. 네 곳 모두 에이전트가 도구를 불러서 읽었다는 표시예요.

이 넷 가운데 하나만 빠져도 추천안은 성립하지 않아요. 원인 후보가 없으면 무엇을 확인할지 모르고, 측정값이 없으면 후보를 가를 수 없죠. 납기를 모르면 생산을 줄이는 안이 얼마나 손해인지 따질 수 없고, SOP가 없으면 조치를 어떻게 수행할지 적을 수가 없습니다.

판단에 쓰는 정보가 여러 곳에 흩어져 있다는 문제, 앞에서 본 적이 있죠. 에이전트는 그 흩어진 정보를 도구로 모아서 추천안 하나로 엮은 겁니다.

[SLIDE 31] 네 정보원이 답하는 물음

그런데 네 정보원은 답하는 물음이 서로 다릅니다. 물음에 맞지 않는 곳을 읽으면 어떻게 될까요? 답이 없거나, 낡은 답을 얻게 돼요.

카드를 하나씩 볼게요. 온톨로지는 무엇과 무엇이 어떤 관계인지에 답합니다. 설비, 증상, 원인, 조치의 관계를 담고 있죠. 시계열 DB는 그 시각의 값이 얼마였는지에 답해요. 센서의 측정값이 시간 순서로 쌓여 있습니다.

업무 DB는 정비, 재고, 납기가 어떤 상태인지에 답합니다. 업무 시스템의 기록이 들어 있어요. 마지막 SOP는 조치를 어떻게 수행하는지에 답하고요. 단계와 승인 역할, 근거가 되는 문서의 절을 정해 둡니다.

물음을 엉뚱한 곳에 던지면 이렇게 됩니다. HYD에서 유온의 현재 측정값을 온톨로지만으로 찾으려 하면, 필요한 값이 없어요. 반대로 원인 후보가 뭔지를 시계열 DB에 물어도 답이 없고요.

처음에 말한 낡은 답은 언제 생길까요? 계속 바뀌는 값을 잘 바뀌지 않는 쪽에 적어 두고 읽을 때예요. 예를 들어 재고 수량을 문서에 적어 두었다면, 문서가 개정될 때까지 그 숫자는 그대로겠죠. 그사이 실제 재고는 계속 바뀌는데요. 그래서 지식 쪽에는 값이 아니라 값을 읽을 자리를 적어 둔다고 했던 겁니다.

카드마다 붙은 꼬리표도 보세요. 그 정보원의 내용이 바뀌는 때입니다. 온톨로지와 SOP는 문서 개정이나 지식 검토를 거쳐 갱신합니다. 시계열 DB는 계속 바뀌고, 업무 DB는 업무가 일어날 때마다 바뀌죠. 이 차이는 뒤에서 지식과 값을 가를 때 다시 씁니다.

[SLIDE 32] 지식 그래프에서 읽는 것: 원인 후보와 조치 방법

네 곳에서 실제로 뭘 읽었는지 하나씩 볼게요. 그 전에 기억해 둘 게 있어요. 네 정보원 가운데 온톨로지와 SOP는 같은 지식 그래프에 들어 있습니다. 지금까지 함께 만들어 온 바로 그 그래프예요.

화면 아래 가운데 진한 원이 고장 유형, 냉각 성능 상실입니다. 위쪽 원인 셋에서 화살표가 이 고장 유형으로 모이죠. 쿨러 핀 오염, 팬 구동 불량, 주변 온도 상승, 이 셋이 이 고장을 일으킬 수 있다는 관계예요. 그리고 오른쪽 아래로는 고장 유형에서 조치 방법으로 가는 관계가 있습니다. 이 고장을 완화하는 방법이 팬 최대 운전이라는 뜻이죠.

선 옆에 작게 적힌 게 관계 이름이에요. 위쪽 세 선은 일으킨다는 관계, 오른쪽 아래 선은 완화한다는 관계입니다. 진단 도구는 고장 유형에서 원인 관계를 거슬러 올라가서 후보 셋을 얻었어요. 완화 관계를 따라가서는 조치 방법을 얻었고요.

이 그래프가 왜 여기서 쓸모 있을까요? 경보는 어떤 고장이 의심된다는 것까지만 알려 줘요. 그런데 관계를 거슬러 가면 확인해야 할 원인 목록이 바로 나옵니다. 에이전트가 원인 후보를 짐작으로 늘어놓을 필요가 없는 거죠. 게다가 이 관계들은 매뉴얼에서 뽑아 사람이 검토하고 등록한 지식이에요. 그래서 에이전트가 내놓는 원인 후보는 검토를 거친 지식 안에서만 나옵니다.

원인 이름표의 둘째 줄도 보세요. 원인마다 사전 확률이 적혀 있죠. 문서에서 지식을 뽑을 때 봤던 값으로, 매뉴얼이 이 원인을 얼마나 흔하다고 보는지를 옮긴 거예요. 셋 가운데 쿨러 핀 오염이 가장 흔합니다.

그래도 한 가지 짚어 둘게요. 흔하다는 게 이번 사건의 원인이라는 뜻은 아니에요. 그래프가 알려주는 건 후보와 그 흔한 정도까지입니다. 셋 가운데 어느 원인이 이번 사건에 맞는지는 그래프만 봐서는 알 수 없어요. 그걸 가르려면 값을 읽어야 합니다.

[SLIDE 33] 시계열 DB에서 읽는 것: 판단하는 순간의 측정값

그 값을 읽는 곳이 시계열 DB예요. 화면 왼쪽은 원인 진단이 확인한 증거 한 건입니다. 여기서 읽은 값 하나를 자세히 들여다볼게요.

첫 줄은 읽은 방법입니다. 최근 30초 동안 냉각 효율의 평균을 읽으라는 거죠. 이 방법은 에이전트가 지어낸 게 아니에요. 지식 그래프의 증거에 적혀 있던 그대로입니다. 앞에서 원인 진단의 물음에는 세 값뿐이었다고 했죠. 무엇을 어떻게 읽을지는 바로 여기, 증거 쪽에 적혀 있었던 겁니다.

구간 평균은 한 번 튄 값의 영향을 줄여서 볼 때 쓸 수 있어요. 반면 짧은 이상을 놓칠 수도 있죠. 그래서 언제나 평균이 더 좋은 것은 아닙니다. 이 사례에서는 등록된 증거 기준이 최근 30초 평균을 요구하므로 그 기간과 집계 방식을 맞춰 읽습니다.

그 아래 측정값은 약 46퍼센트, 기준은 70퍼센트 미만입니다. 앞의 값은 시계열 DB에서 읽었고, 뒤의 기준은 지식 그래프에서 가져왔어요. 이 측정값은 기준보다 낮으므로 해당 증거 조건을 만족합니다. 원인 전체의 판정은 다른 증거와 함께 봐야 하고요.

맨 아래에는 관측 범위에 관한 정보도 붙어 있습니다. 표본이 30개였다는 기록이 보여요. 다만 개수만으로 매초 빠짐없이 들어왔다고 단정하면 안 됩니다. 관측 시각과 비어 있는 구간도 확인해야 하죠. 평균값을 얻었더라도 관측 공백이 크면 그 구간을 대표한다고 보기 어렵기 때문입니다. HYD의 진단도 이런 데이터 품질 조건을 함께 확인합니다.

[SLIDE 34] 업무 DB에서 읽는 것: 설비 밖의 사정

다음은 업무 DB예요. 납기나 남은 수량은 설비에 달린 센서로는 알 수가 없죠. 이런 값은 업무 DB에 있습니다.

화면 왼쪽은 앞에서 본 다른 실행의 기록에서 꺼낸 생산오더 조회예요. 위쪽이 물음인데, 에이전트는 설비 이름 하나만 넣었습니다. 그 아래가 돌려받은 결과고요. 오른쪽 첫 설명을 보면, 값이 온 업무 시스템을 표시한 칸이에요. 생산 일정을 관리하는 MES입니다.

둘째 설명은 판단에 쓰는 값들을 모아 놓은 칸이에요. 이 설비에 걸린 생산오더는 납기까지 약 6시간이 남았고, 아직 만들어야 할 수량도 꽤 남아 있었습니다. 같은 방식으로 품질 쪽에서는 출하를 기다리는 로트를 읽었고요. 오른쪽 셋째 설명이 그거예요. 이 로트는 품질 쪽 재료입니다. 유온이 높은 채로 이어지면 출하를 앞둔 제품에 품질 문제가 생길 위험이 커지는데, 그 위험을 따지는 데 쓰여요.

쿨러 사건의 실행에서는 이 조회를 따로 부르지 않았죠. 대안 평가가 같은 생산오더의 납기를 평가 안에서 읽었어요. 그 값이 조치를 견줄 때 쓰였습니다. 부하를 낮추는 조치는 유온을 더 내리지만 생산이 줄어요. 납기까지 6시간도 안 남은 상황이라, 그 손실이 성과 비교에서 점수로 드러난 거죠. 만약 유온을 많이 내리는 정도만 비교했다면 그 효과가 큰 안을 앞세웠겠죠. 실제로는 그 안이 공동 2순위였죠. 설비 밖의 사정이 설비 조치의 순위를 바꾼 겁니다.

[SLIDE 35] HYD의 데이터베이스 접근: 읽기 전용 도구 경유

그럼 에이전트에게 데이터베이스를 자유롭게 열어 주어야 할까요? HYD는 그 사이에 도구를 두었습니다. 화면의 에이전트는 도구에 물음을 보내고, 도구가 제한된 권한으로 읽어 온 결과를 받습니다. 데이터베이스를 직접 연결하는 구성도 가능하지만, 여기서는 접근 경로를 도구로 관리하는 방식을 택한 거예요.

가운데 자물쇠가 그 제한을 나타냅니다. 조회 도구는 읽기 전용 계정을 사용하므로, 잘못된 조회 요청이 데이터를 수정하거나 지우는 작업으로 실행되지 않게 합니다. 조회 결과의 양과 실행 시간에도 한도를 둡니다. 화면의 200행과 5초는 이 시스템의 SQL 조회 설정이고, 모든 도구에 공통인 숫자는 아니에요.

결과의 행 수를 제한하는 것과 데이터베이스가 계산할 양을 제한하는 것은 다릅니다. 한 줄짜리 합계를 얻으려고도 많은 데이터를 읽을 수 있거든요. 시간 제한 같은 장치를 함께 두어 자원 사용을 줄이지만, 조회가 항상 가볍고 안전하다고 보장할 수는 없습니다. 질문에 맞는 기간과 범위를 정하는 일도 필요해요.

또한 읽기 도구의 제한을 에이전트의 모든 행동과 혼동하면 안 됩니다. 추천안 제출 도구는 기록을 남깁니다. 설비 제어 명령은 또 별도의 권한과 승인 절차를 거치고요. 무엇을 읽을 수 있고 무엇을 쓸 수 있는지는 도구와 업무마다 정해야 합니다.

정보를 얻는 길이 정해졌습니다. 다음에는 제대로 읽어 온 정보라도 잘못 해석할 수 있는 지점, 기준과 측정값의 구분을 살펴보겠습니다.

[SLIDE 36] 지식과 현재값의 구분

앞에서 정보원마다 붙은 꼬리표를 보면서, 잘 바뀌지 않는 지식과 계속 바뀌는 값이 나뉜다고 했죠. 그 둘을 가르는 게 다섯 번째 단원, 지식과 현재값의 구분입니다. 제대로 읽은 정보라도, 에이전트가 이 둘을 섞어 쓰면 어떤 일이 생길까요? 섞이는 꼴부터 보고, 그걸 막는 장치 두 가지를 차례로 보겠습니다.

[SLIDE 37] 지식과 현재값을 섞는 두 가지 꼴

지식과 현재값이 섞이는 꼴은 두 가지예요. 화면의 두 칸에 틀린 문장의 예를 하나씩 놓았습니다. 따옴표 안이 그 틀린 문장이에요.

왼쪽 문장은 냉각 효율이 70 %라고 말합니다. 그런데 70은 매뉴얼에 적힌 판별 기준이죠. 잰 값이 아니에요. 실제로 잰 값은 약 46퍼센트였습니다. 기준을 측정값처럼 말한 경우예요.

이 혼동이 판단을 어떻게 바꾸는지 따라가 볼까요? 기준은 70 미만이었죠. 이 70을 잰 값으로 알고 기준에 대면, 70은 70 미만이 아니니까 냉각 효율이 낮다는 증거는 맞지 않는 걸로 판정됩니다. 쿨러 핀 오염을 뒷받침하던 증거 하나가 엉뚱하게 사라지는 거예요.

오른쪽 문장은 유온이 약 56도이라서 한계까지 여유가 있다고 말합니다. 그런데 그 값은 경보가 난 순간에 읽은 값이에요. 운전원이 승인하던 순간의 유온은 62도 가까이로 올라 있었습니다. 한 번 읽은 값을 한참 뒤에도 지금의 값처럼 쓴 경우죠. 그사이 한계까지의 여유는 꽤 줄어 있었어요. 여유가 있다고 믿으면 덜 급한 조치를 골라도 된다는 판단으로 흐르기 쉽습니다.

두 문장 다 숫자가 들어 있어서 그럴듯하게 읽힙니다. 문제는 읽는 사람이 알아차리기 어렵다는 거예요. 그래서 이 혼동은 사람이 잡아내길 기다릴 게 아니라, 에이전트가 처음부터 하지 않도록 막아야 합니다.

[SLIDE 38] 혼동을 막는 작업 규칙: 추측 대신 도구로 확인한다

그걸 막는 첫째 장치가 작업 규칙입니다. 에이전트에게는 추측하지 말고 도구로 확인하라는 규칙이 주어져요. 무엇을 어디서 얻을지가 화면처럼 세 갈래로 정해져 있습니다.

첫째 카드는 지식이에요. 원인 후보, 증거의 기준, 조치 방법은 지식 그래프를 조회해서 얻습니다. LLM만으로는 이 공장의 기준을 몰라서 일반론으로 대신 답한다고 했죠. 그 빈자리를 메우는 게 이 카드예요. 모델이 원래 알고 있는 일반 지식으로 기준을 말하지 않고, 이 공장의 기준을 그래프에서 꺼내 씁니다. 둘째 카드는 지금의 값이고요. 센서 값, 납기, 재고는 시계열 DB와 업무 DB에서 그때그때 읽어요. 기억해 둔 값을 쓰지 않는 거죠. 이 카드의 마지막 줄도 보세요. 값마다 어디서 어떻게 읽었는지가 함께 붙습니다. 쿨러 사건에서도 유온에는 시계열 DB의 최신값이라는 출처가, 긴급 오더의 남은 시간에는 MES라는 출처가 붙어 있었어요. 그래서 나중에 이 값이 어디서 왔는지를 따져 볼 수 있죠.

셋째 카드는 규칙 판정입니다. 어떤 조치가 제외되는지, 감점이나 경고가 붙는지, 이건 모델이 해석하지 않아요. 판단 도구가 계산한 결과를 씁니다.

규칙 판정은 왜 도구에 맡길까요? 모델에게 규칙을 해석하게 하면, 같은 상황에서도 답이 달라질 수 있거든요. 판단 도구는 같은 판본의 규칙에 같은 입력을 넣으면 같은 방식으로 계산합니다. 그래야 판정을 믿을 수 있고, 다시 확인할 수도 있죠.

물론 규칙을 글로 주었다고 모델이 반드시 지키는 것은 아닙니다. 도구의 권한 제한, 데이터 상태 검사, 결과 검토가 함께 필요해요. 지금 본 것은 어떤 출처를 써야 한다는 지시이고, 다음은 그 출처에서 읽은 값의 상태를 검사하는 장치입니다.

[SLIDE 39] 신선도: 관측된 값이 얼마나 오래됐는가

방금 조회에 성공했다고 해서 지금의 상태를 읽었다고 볼 수 있을까요? 센서가 멈췄다면 데이터베이스에는 오래전 값이 마지막 값으로 남아 있을 수 있습니다. 그래서 신선도는 조회 버튼을 누른 때가 아니라, 값이 관측되어 기록된 시각이 판단 시점에서 얼마나 오래됐는지를 봅니다.

화면의 굵은 세로선은 판단 시점이고, 그 왼쪽 띠가 허용 범위예요. HYD의 이 검사에서는 최신 유온의 관측 시각이 60초 안쪽인지 봅니다. 60초는 이 시스템의 설정이지 모든 설비의 공통 기준은 아닙니다. 공정이 얼마나 빨리 변하고 어떤 판단에 쓸 값인지에 맞춰 정해야 해요.

쿨러 사건의 검사 시점에는 최신 유온이 1초도 안 된 값이었습니다. 반대로 60초보다 오래된 값만 남아 있었다면 이 검사를 통과하지 못해요. HYD는 수집 상태와 시각의 이상도 함께 확인합니다. 최신 행을 찾았다는 사실만으로 충분하지 않은 거죠.

검사에 통과한 값도 시간이 지나면 오래됩니다. 조사와 평가가 길어졌다면 중요한 실행 전에 다시 확인할 필요가 있어요. 다만 지금 화면은 한 검사 시점의 결과입니다. 모든 제출과 승인 순간에 자동으로 재검사됐다고 이 기록만으로 말할 수는 없습니다. 실제 업무를 설계할 때는 무엇을 어느 단계에서 다시 확인할지도 정해야 합니다.

[SLIDE 40] 멈춤 조건: 근거 부족 · 호출 실패 · 권한 제한

방금 값이 묵었으면 판단을 보류한다고 했죠. 일을 끝까지 하는 것보다 멈추는 게 맞을 때가 있다는 뜻이에요. 여섯 번째 단원은 바로 그 멈춤 조건입니다. 에이전트는 언제 멈춰야 하고, 멈출 때는 무엇을 내야 할까요?

[SLIDE 41] 멈추지 않는 에이전트가 만드는 결과

먼저 에이전트가 멈추지 않으면 어떻게 되는지부터 볼게요. 냉각 효율을 읽지 못한 상황 하나에서 나올 수 있는 두 결과를 나란히 놓았습니다.

왼쪽은 빈칸을 채워서 제출하는 경우예요. 냉각 효율을 못 읽었는데, 사전 확률이 가장 큰 쿨러 핀 오염을 원인으로 적어 냈습니다. 앞에서 본, 가장 흔하다는 그 값만 믿고 적은 거죠. 그런데 이 추천안, 모양은 값을 읽었을 때와 똑같아요. 담당자는 확인된 원인이라고 믿고 승인하겠죠. 원인이 틀렸더라도 겉으로는 드러나지 않습니다. 문제는 원인이 다르면 맞는 조치도 달라진다는 거예요. 쿨러 핀이 오염된 거라면 핀을 세척해야 하지만, 팬이 제대로 돌지 않는 거라면 팬 쪽을 점검해야 하죠. 틀린 원인 위에 세운 추천안은 엉뚱한 정비로 이어질 수 있습니다.

오른쪽은 멈추고 이유를 내는 경우예요. 원인을 적지 않고 보류합니다. 그리고 무엇을 확인하지 못했는지를 적어 내죠. 그러면 담당자는 값이 들어온 뒤에 다시 평가하게 할 수 있어요. 모르는 게 모른다는 모습 그대로 드러나는 겁니다.

멈춘다는 건 일을 못 한 게 아니에요. 승인하는 사람이 없는 근거를 믿고 결정하지 않게 하는 일입니다. 그래서 어떤 때 멈출지를 미리 정해 둬요.

[SLIDE 42] 멈춤 조건 세 가지

에이전트가 멈춰야 하는 대표적인 상황은 셋입니다. 화면 왼쪽 에이전트에서 세 갈래로 갈라지죠. 위에서부터 볼게요.

위 갈래는 근거 부족이에요. 값을 얻지 못했거나, 얻은 값이 어느 원인의 기준에도 맞지 않는 경우입니다. 가운데 갈래는 도구 호출 실패고요. 다시 시도해도 도구가 오류를 돌려주는 경우죠. 아래 갈래는 권한 제한입니다. 일을 끝내려면 허용되지 않은 도구가 필요한 경우예요.

갈래 끝마다 그 상황을 뜻하는 그림이 있어요. 근거 부족에는 값이 빈 저장소, 호출 실패에는 가위표가 붙은 도구, 권한 제한에는 자물쇠가 그려져 있죠.

세 갈래 모두 멈춤 표지를 지납니다. 에이전트는 지어내지 않고 멈추는 거죠. 그리고 오른쪽의 사람에게 모여요. 사람은 다시 평가하게 하거나, 에이전트의 질문에 답하거나, 작업을 닫습니다. 오른쪽 사람 이름표의 둘째 줄이 그 세 가지예요.

세 상황의 공통점은, 에이전트가 근거를 갖고 끝까지 갈 수 없다는 거예요. 그럴 때 할 일은 그럴듯한 결과를 만드는 게 아닙니다. 이유와 함께 사람에게 넘기는 거죠.

[SLIDE 43] 근거의 세 가지 상태: SUPPORTED · UNKNOWN · UNSUPPORTED

세 조건 가운데 근거 부족부터 자세히 볼게요. 근거 부족에는 사실 서로 다른 두 경우가 들어 있어요. 그래서 근거의 상태를 화면처럼 셋으로 나누어 적습니다.

왼쪽 카드는 근거가 지지하는 상태예요. 필요한 값을 읽었고 등록된 증거 기준에 맞았습니다. 냉각 효율이 약 46퍼센트로 70퍼센트 미만이었다는 것은 그중 한 조건이에요. 쿨러 핀 오염이라는 후보의 판정은 팬 진동을 비롯한 필요한 증거와 함께 봅니다. 지지된다는 말은 이 모델과 기준 안에서의 판정이지, 현장의 모든 가능한 원인을 배제했다는 뜻은 아닙니다.

가운데 카드는 확인하지 못한 상태입니다. 조회에 실패했거나 값이 비어서, 맞는지 틀린지를 말할 수가 없어요. 결정표를 다룰 때도 값을 얻지 못한 조건을 통과로 치지 않았죠. 여기서도 같은 생각이에요. 모르는 것을 맞다고도, 틀리다고도 하지 않고 모른다고 적는 겁니다.

오른쪽 카드는 맞는 근거가 없는 상태고요. 값은 읽었는데, 조회한 근거가 모두 기준에 맞지 않았습니다. 등록된 원인 가운데 맞는 게 하나도 없는 거죠.

가운데와 오른쪽을 굳이 나누는 까닭은, 사람이 이어서 할 일이 다르기 때문이에요. 확인하지 못한 경우는 필요한 값이 들어오면 다시 평가할 수 있습니다. 맞는 근거가 없는 경우는 다르죠. 지식 그래프에 등록되지 않은 원인일 수 있어요. 그러면 사람이 직접 살펴야 합니다.

[SLIDE 44] 보류의 형식: 상태 · 이유 · 증거

상태를 나눴으면, 보류할 때 내는 형식도 정해져 있어야겠죠. 화면 왼쪽이 보류할 때 에이전트가 내는 객체예요. 작업 지시에 적혀 있는 형식 그대로인데, 항목이 세 개뿐입니다.

첫째 칸은 상태예요. 방금 본 확인하지 못함과 맞는 근거 없음 가운데 하나를 적습니다. 둘째 칸은 이유고요. 무엇을 확인하지 못했는지를 사람이 읽을 문장으로 적어요. 예를 들면 유온의 최신 값이 60초보다 오래되었다, 이런 식이죠. 보류된 작업은 결국 사람이 받으니까, 사람이 읽고 바로 무엇을 해야 할지 알 수 있는 문장이어야 합니다.

셋째 칸은 그때까지 조회해서 얻은 실제 근거입니다. 멈췄다고 해서 그때까지 얻은 걸 버리지 않아요. 그래야 사람이 다시 평가를 시킬 때, 무엇은 이미 확인됐고 무엇이 비었는지를 한눈에 볼 수 있죠.

화면 아래도 보세요. 원인, 고장 유형, 판단 같은 결과 항목은 보류와 함께 낼 수 없습니다. 섞어서 내면 거절돼요.

섞어 내기를 막아 두지 않으면 어떻게 될까요? 일부만 채운 결과가, 절반만 확인된 추천안으로 승인 화면에 올라가게 됩니다. 그래서 보류는 결과를 하나도 채우지 않고, 이유와 근거만 넘기는 거예요.

[SLIDE 45] 조회 실패 · 일치하는 기록 없음 · 수량 0

도구가 오류를 돌려준 경우와 조회는 됐지만 찾은 행이 없는 경우도 구분해야 합니다. 둘을 같은 빈칸으로 처리하면, 확인하지 못한 일을 없는 일로 바꿔 버릴 수 있어요.

예비품 재고를 조회한다고 해 볼게요. 왼쪽은 조회에 성공했지만 조건에 맞는 기록이 0행인 경우입니다. 이때 알 수 있는 것은 그 조회 범위 안에서 일치하는 기록을 찾지 못했다는 사실이에요. 실제 창고의 재고가 없다고 바로 단정할 수는 없습니다. 창고나 품목 조건을 잘못 넣었거나, 해당 재고가 아직 등록되지 않았을 수도 있죠.

재고 행을 찾았고 그 행의 수량이 0인 경우는 또 다릅니다. 조회한 대상의 기록상 수량이 0이라는 답을 받은 거예요. 어느 창고와 어느 시점의 기록인지까지 맞아야 질문의 답으로 쓸 수 있습니다.

오른쪽은 조회 자체가 실패한 경우입니다. 수량도, 조건에 맞는 기록의 유무도 확인하지 못했어요. 이 실패를 0으로 바꾸면, 읽지 못한 재고를 없다고 믿고 새로 발주할 수도 있습니다. 그래서 실패, 일치하는 기록 없음, 기록된 수량 0을 각각 그대로 남겨야 해요. 필수 값을 끝내 확인하지 못했다면 판단을 보류합니다.

[SLIDE 46] 재시도: 반복의 영향과 한도

방금 다시 시도해도 끝내 실패하면 실패했다고 그대로 적는다고 했죠. 그럼 끝내라는 건 언제일까요? 에이전트가 끝없이 다시 시도할 수는 없어요. 다시 시도하는 데는 한도가 있습니다. 화면의 카드 셋이 그 한도고, 카드 아래 꼬리표에 한도나 그걸 정하는 쪽이 적혀 있어요.

첫째 카드는 도구를 다시 부르는 일이에요. 앞에서는 결과가 너무 커서 읽는 방법을 바꾸겠다는 기록을 봤죠. 다시 시도할 때는 먼저 읽기인지 쓰기인지 구별해야 합니다. 조회를 다시 하는 것과 추천안이나 발주를 다시 제출하는 것은 영향이 다르거든요. 제출 응답이 늦었다고 해서 저장까지

실패한 것은 아닐 수 있어요. 같은 요청을 무작정 보내면 두 건이 생길 수 있습니다. 기존 처리 여부를 확인하거나 같은 요청의 중복 반영을 막는 장치가 필요합니다.

둘째 카드는 조회문 수정이에요. 질문에 답하는 에이전트가 조회문을 직접 써서 실행했던 것, 앞에서 봤죠. 그렇게 직접 쓴 조회문이 오류를 내면 고쳐서 다시 실행할 수 있는데, 두 번까지 시도하라고 작업 지침에 적혀 있습니다. 이 숫자는 모델에게 주는 지시이고, 그 자체가 서버의 강제 차단을 뜻하지는 않아요. 고칠 때 지키는 게 하나 있어요. 오류가 난 부분만 고치고, 무엇을 세고 무엇을 거를지는 바꾸지 않습니다. 오류를 없애려고 묻는 내용 자체를 바꿔 버리면, 오류는 사라져도 엉뚱한 물음에 대한 답을 얻게 되니까요. 두 번을 고쳐도 안 되면, 방금 본 대로 실패했다고 그대로 적습니다.

셋째 카드는 실행 시간이에요. 확인한 실행 설정에는 한 번의 실행 전체에 30분의 제한이 걸려 있습니다. 꼬리표에는 초로 적혀 있죠. 이것도 이 시스템에 정해 둔 값이에요. 이 시간을 넘기면 작업은 멈춘 채로 남고, 사람이 닫아야 해요. 시간이 다 된 경우도 멈춤 조건과 같은 자리에 있는 셈입니다. 근거를 갖고 끝까지 가지 못했으니, 결과를 지어내지 않고 멈추는 거죠.

한도는 왜 둘까요? 에이전트가 같은 실패를 끝없이 되풀이한다고 생각해 보세요. 결과는 안 나오고 시간만 흐르겠죠. 사람은 그동안 아무것도 알 수가 없고요. 그래서 한도를 넘긴 작업은 멈춰서 사람에게 넘어갑니다.

[SLIDE 47] 권한 제한: 허용 목록 밖의 도구

마지막 멈춤 조건은 권한 제한이에요. 출발점은 화면 첫 칸의 허용 목록입니다. HYD는 에이전트의 도구 호출을 이 허용 목록으로 제한합니다.

둘째 칸을 보세요. 에이전트가 목록 밖의 도구를 부르려고 하면, 그 호출은 실행되지 않고 권한 요청이 됩니다. 셋째 칸, 그 요청은 다시 질문이 되어 사람에게 올라가요. 에이전트가 일을 하려면 꼭 필요하다고 판단했더라도 마찬가지입니다.

마지막 칸은 에이전트에게 주는 지시의 한 문장이에요. 답변은 실행 권한을 바꾸지 않는다. 사람이 그 질문에 답하더라도, 그 답으로 에이전트의 권한이 넓어지지는 않는다는 뜻입니다.

왜 답변으로 권한이 바뀌면 안 될까요? 무엇을 부를 수 있는지는 에이전트를 정의할 때 미리 정해 둔 거예요. 맡긴 업무에 필요한 기능과 권한을 미리 검토해서 등록하는 것이죠. 실행 중에 오간 질문과 답으로 그게 바뀐다면, 미리 정해 둔 목록은 의미가 없어지겠죠. 권한을 넓히는 일은 에이전트의 정의를 고치는 일이지, 일하는 도중의 대화로 할 일이 아닌 겁니다.

여기서 에이전트가 해서는 안 되는 일이 하나 있어요. 허용된 도구를 이리저리 엮어서, 막힌 일을 돌아가는 겁니다. 지시에도 사람에게 묻는 질문을 도구 거절이나 승인 우회에 쓰지 말라고 적혀 있어요.

세 조건을 다 봤으니 이 단원의 물음에 답해 볼까요? 에이전트는 언제 멈추고 무엇을 내는가. 근거가 모자랄 때도, 호출이 끝내 실패할 때도, 권한 밖의 일이 필요할 때도 할 일은 같습니다. 결과를 채우지 않고 멈춰서, 이유와 근거를 붙여 사람에게 넘기는 거죠.

[SLIDE 48] 구조화 출력과 실행 기록

멈출 때 내는 보류에는 정해진 형식이 있었죠. 멈추지 않고 끝까지 간 결과에도 형식이 있습니다. 그걸 다루는 게 마지막 이론 단원, 구조화 출력과 실행 기록이에요. 사람이 추천안을 믿고 확인하려면, 결과와 과정이 어떤 모양으로 남아야 할까요? 결과를 정해진 형식으로 내는 구조화 출력부터 보고, 일한 과정을 남기는 실행 기록, 그리고 확인된 원인과 가설을 가르는 것까지 보겠습니다.

[SLIDE 49] 구조화 출력: 결과 객체의 다섯 항목

결과 쪽부터 볼게요. 화면 왼쪽은 쿨러 사건에서 에이전트 단계가 끝날 때 남긴 결과의 뼈대예요. 종료 장면에서 작업 공간에 썼다던 그 결과 파일이죠. 다섯 항목을 가진 JSON 객체 하나입니다. 문서에서 지식을 뽑을 때 썼던 구조화 출력이 여기서도 쓰인 거죠.

맨 위 두 항목은 원인과 고장 유형이에요. 이름이 아니라 지식 그래프의 식별자로 적습니다. 셋째 항목은 진단 안내고요. 진단의 결과와 권장 조치를 담고 있어요.

넷째는 판단 항목입니다. 오른쪽 설명에서 강조한 항목이죠. 추천하는 조치, 그 이유의 설명, 후보의 순서가 들어 있어요. 앞에서 추천안의 표로 본 추천, 이유, 뒤진 대안이 바로 이 안에 있는 거죠. 이 가운데 설명은 승인하는 사람이 읽는 부분입니다. 마지막 항목은 판단 제출 도구에서 돌려받은 판단 번호예요. 판단 제출로 낸 추천안과 이 결과가 이 번호로 이어집니다.

여기서 헷갈리기 쉬운 게 하나 있어요. 오른쪽 셋째 설명의 끝줄처럼, 항목 칸에는 식별자를 적지만 설명 문장에는 식별자 대신 팬 최대 운전처럼 이름을 씁니다. 칸은 다음 시스템이 받아서 정확히 찾아가야 하니까 식별자고, 문장은 사람이 읽으니까 이름인 거죠.

어떤 항목이 있어야 하는지는 작업마다 미리 정해 두는데, 이걸 출력 계약이라고 해요. 승인 화면은 판단 항목에서 추천과 설명을 꺼내 보여 주고, 뒤의 단계는 판단 번호로 이 판단을 가리킵니다. 항목이 정해져 있어야, 받는 쪽이 어디서 무엇을 꺼낼지 알 수 있는 거죠.

[SLIDE 50] 출력 형식의 검사와 교정

그럼 형식이 어긋나면 어떻게 될까요? 에이전트가 낸 결과는 시스템이 먼저 검사합니다. 형식이 맞아야 다음 단계로 넘어가요.

화면의 네 칸을 따라가 볼게요. 첫 칸에서 에이전트가 결과 객체를 냅니다. 둘째 칸에서 시스템이 정해진 항목과 맞는지를 보고요. 항목이 빠졌거나 모양이 어긋나면 셋째 칸으로 가요. 출력 형식 검사에서 거절됐다는 알림이, 이유와 함께 에이전트에게 돌아가죠.

에이전트는 같은 실행 안에서 고쳐서 다시 냅니다. 내용은 확인됐고 형식만 어긋난 경우라면, 얻어 둔 내용을 정해진 항목에 맞춰 고치는 겁니다. 형식을 맞추려고 새로운 값을 지어 넣으면 안 되겠죠. 필요한 사실 자체가 빠진 경우는 단순한 모양 수정으로 해결할 수 없습니다. 이 시스템의 형식 교정은 최대 두 번이며, 통과하면 승인할 담당자에게 넘어갑니다.

그런데 이 검사가 보는 건 형식이에요. 항목이 다 있는지, 모양이 맞는지를 봅니다. 형식이 맞는다고 내용까지 옳다는 뜻은 아니죠. 내용이 옳은지는 다른 방법으로 확인해야 합니다.

[SLIDE 51] 실행 기록 한 줄에 남는 것

내용을 확인하는 방법, 그게 과정의 기록이에요. 결과와 함께 과정도 남습니다. 에이전트가 일하는 동안 말과 도구 호출이 시각 순서로 한 줄씩 남는데, 이걸 실행 기록이라고 해요. 화면 위의 상자는 그 실행 기록을 보여 주는 포털의 처리 기록 화면입니다. 앞에서 처리 기록이라고 부르며 본 줄들이 바로 이 실행 기록이었던 거죠.

그 가운데 진하게 표시한 셋째 줄이 도구 호출 한 건입니다. 위 상자의 그 줄을 아래로 끌어내려서 다섯 칸으로 펼쳤어요. 왼쪽부터 언제, 어느 도구에, 무엇을 물어서, 무엇을 받았고, 얼마나 걸렸는지입니다.

앞에서 본 원인 진단 호출도 이 다섯 항목으로 읽을 수 있습니다. 숫자와 식별자를 모두 읽기보다, 지금 확인하려는 값이 어느 입력에서 나와 어떤 결과로 돌아왔는지 연결해 보세요.

여기서 중요한 건 가운데 두 칸이에요. 입력과 출력이 모두 남는다는 점입니다. 그래서 추천안에 적힌 값이 어느 호출에서 왔는지를 찾아갈 수 있어요.

같은 경보를 처리한 두 실행이 서로 다른 길로 일했던 것, 기억하죠. 같은 지침이라도 실행마다 부른 도구와 순서가 달라질 수 있어요. 그래서 결과만 보고는 이번 실행이 어떻게 일했는지 알 수 없고, 실행마다 이 기록이 남아야 하는 겁니다.

[SLIDE 52] 실행 기록의 쓰임: 결과에서 호출로 거슬러 올라가기

이 기록은 언제 쓸까요? 결과가 이상해 보일 때입니다. 화면의 네 단계처럼, 결과에서 출발해 호출로 거슬러 올라가는 거예요.

첫째, 추천안에서 의심스러운 값을 고릅니다. 둘째, 그 값을 돌려준 호출을 기록에서 찾아요. 출력이 그 값과 같은지 확인하는 거죠. 셋째, 그 호출의 입력을 봅니다. 엉뚱한 설비를 넣지 않았는지, 틀린 원인을 넣지 않았는지요.

먼저 도구의 출력과 최종 설명이 같은 뜻인지 봅니다. 모델이 단위를 바꾸거나 다른 호출의 값을 섞었을 수도 있어요. 입력도 질문에 맞고 출력도 그대로 옮겼다면, 그때 도구의 계산과 원천 데이터까지 살펴봅니다. 호출 사이의 진행 설명은 에이전트가 밝힌 의도를 읽는 보조 자료예요. 그 설명만으로 실제 처리나 오류의 원인을 확정하지는 않습니다.

쿨러 사건에 대어 볼까요? 추천안의 원인, 쿨러 핀 오염이 의심스럽다고 해 봅시다. 그 값을 돌려준 호출은 원인 진단이죠. 기록에서 그 호출의 받음을 열어, 원인 1순위가 정말 쿨러 핀 오염인지 봅니다. 맞다면 물음으로 가요. 설비가 HYD-01이 맞는지, 경보 번호가 이 경보가 맞는지. 입력과 반환값, 최종 설명의 연결까지 맞는다면 진단 도구가 읽은 증거와 값 쪽을 더 확인합니다.

요청 해석 장면에서, 에이전트가 진행 설명을 남긴다고 했죠. 그 말이 여기서 쓰이는 거예요. 결과만 있으면 틀렸다는 것밖에 모릅니다. 하지만 과정이 남아 있으면, 어디서 틀렸는지를 찾을 수 있어요.

[SLIDE 53] 근거 제시: 식별자의 인용

과정의 기록과는 별도로, 결과 안에도 근거가 들어 있어야 합니다. 에이전트는 모든 판단에 지식 그래프의 식별자를 인용하게 되어 있어요.

화면의 카드 셋이 쿨러 사건의 추천안에 인용된 식별자입니다. 원인은 쿨러 핀 오염, 고장 유형은 쿨러 냉각 성능 상실, 조치 방법은 팬 최대 운전이죠. 카드마다 붙은 꼬리표는 식별자의 앞머리예요. 원인, 고장 유형, 조치 방법이 각각 다른 앞머리를 갖습니다.

식별자는 다음 시스템이 받아 쓰기 위한 것만이 아니에요. 근거로 쓸 때도 이름과는 다르죠. 쿨러 핀 오염이라는 이름만 있으면, 그게 등록된 지식인지 모델이 지어낸 말인지 구별할 수가 없어요. 식별자가 있으면 그 노드를 찾아갈 수 있습니다. 조치 방법이라면 출처인 매뉴얼의 절까지 확인할 수 있고요. 강조한 셋째 카드에 붙은 출처가 바로 매뉴얼의 즉시 완화 조치 절이에요. 문서에서 지식을 등록할 때 출처를 함께 남겼던 게, 여기서 에이전트의 근거로 쓰이는 겁니다.

식별자가 적혀 있다는 사실 자체가 근거의 정확성을 보장하지는 않습니다. 실제 노드가 존재하는지, 이번 설비와 연결되는지, 인용한 절이 지금의 주장까지 뒷받침하는지 확인해야 해요. 식별자는 그 확인을 시작할 주소 역할을 합니다.

[SLIDE 54] 근거의 대조: 적어 낸 근거가 호출 기록에 있는가

그렇다고 에이전트가 근거를 적었다는 것만으로 충분할까요? 아닙니다. 근거 자체를 꾸며 낼 수도 있으니까요. 그래서 시스템이, 적어 낸 근거를 실제 호출 기록과 대조합니다.

질문에 답하는 경우로 볼게요. 앞에서 완제품 재고를 물었을 때, 답 아래에 조회문이 근거로 붙어 있었죠. 시스템은 바로 그 조회문을 호출 기록과 대어 봅니다. 왼쪽은 호출 기록에 있는 근거예요. 답에 적힌 조회문이 실제로 실행한 조회와 같습니다. 그러면 화면에 근거 몇 개가 확인됐다고 표시되고, 답으로 보여요.

오른쪽은 호출 기록에 없는 근거입니다. 답에 적힌 조회를 기록에서 찾을 수가 없어요. 모델은 그럴듯한 조회문을 얼마든지 적을 수 있거든요. 실제로 돌려 보지 않았더라도요. 이때 시스템은 그 답을 답으로 보여 주지 않습니다. 답할 수 없음과 그 이유를 보여 주고, 에이전트가 낸 답은 근거가 확인되지 않았다는 표시와 함께 접어 둬요.

실행 기록은 이렇게, 사람이 읽는 데만 쓰이는 게 아니에요. 시스템이 에이전트의 말을 검사하는 데도 쓰입니다. 첫 실습에서 에이전트에게 질문해 보면, 이 표시를 직접 보게 될 거예요.

이 확인 표시의 범위도 구분해야 합니다. 서버가 확인하는 것은 적힌 도구와 조회가 성공한 호출 기록에 대응하는지입니다. 반환된 수량을 답에 정확히 옮겼는지, 조회 기간이 질문에 맞는지까지

자동으로 정답 판정하는 것은 아니에요. 근거가 여러 개라면 각각의 확인 상태도 봅니다. 첫 실습에서도 표시만 보고 끝내지 않고, 실제 반환값과 답의 내용을 대조할 겁니다.

[SLIDE 55] 기준이 지지한 원인과 가설: 판정의 범위

근거를 적을 때 조심할 게 하나 더 있습니다. 두 값이 함께 변했다는 것만으로 원인을 정하면 안 돼요.

화면 왼쪽은 단정하는 경우예요. 유온이 오르는 동안 냉각 효율이 떨어졌습니다. 그래서 쿨러가 원인이라고 적어요. 그런데 팬 구동 불량이나 주변 온도 상승에서도 유온 상승은 나타날 수 있죠. 같은 쿨러 성능 저하 경보에 원인 후보가 셋이었던 게 그래서예요.

오른쪽은 등록된 증거 기준과 비교하는 경우입니다. 냉각 효율은 낮았고, 팬 진동은 이 사례의 기준에서 정상 범위였어요. 캐비닛 온도도 주변 온도 상승을 지지하는 기준에 못 미쳤습니다. 그래서 이 모델에 등록된 후보 중에서는 쿨러 핀 오염이 근거의 지지를 받았어요. 팬 진동이 낮다는 사실만으로 팬의 모든 문제가 없다고 증명한 것은 아닙니다.

화면에서 확인된 원인이라고 부르던 것도 이 판정 범위 안에서 읽어야 합니다. 관측값을 여러 기준과 비교해 가설을 좁힌 것이고, 모든 현장 원인을 확인한 최종 조사와 같지는 않아요. 중요한 조치는 정해진 현장 점검과 승인 절차로 이어져야 합니다.

값을 읽지 못해서, 지우지도 확인하지도 못한 후보가 있다면 어떻게 할까요? 그 후보는 가설이라고 표시합니다. 확인된 원인과 가설을 가르는 것, 이게 에이전트가 근거를 다루는 마지막 장치예요.

이 단원 처음에, 사람이 추천안을 믿고 확인하려면 무엇이 남아야 하느냐고 물었죠. 결과는 정해진 항목으로 내고, 과정은 호출마다 기록으로 남깁니다. 근거는 식별자로 인용해서 기록과 대조하고요. 그리고 확인된 것과 가설을 나눠 적습니다. 이게 다 갖춰져야 사람이 추천안을 확인하고 승인할 수 있어요.

[SLIDE 56~57] 실습: 나의 첫 업무 에이전트 구성하기

모델의 말과 도구의 결과를 구별하고, 근거가 부족할 때 멈추는 기준도 살펴봤습니다. 이 기준을 직접 확인하도록 네 실습을 준비했습니다. 첫 실습에서는 업무 에이전트를 하나 구성해 봅니다. 포털의 에이전트 화면에서 이름과 목표와 역할을 적고, 쓸 수 있는 읽기 도구 가운데 필요한 걸 골라요. 두 에이전트의 정의를 견줄 때 봤던 그 항목들이죠.

구성한 에이전트에게는 질문을 하나 던져 볼 겁니다. 답이 나오면 아래 붙은 근거와 도구 호출 기록을 확인해요. 확인 표시가 있는지 보고, 실제 반환값이 답에 맞게 쓰였는지도 대조합니다. 어떤 자료를 읽었는지와 그 자료로 제대로 답했는지를 함께 보는 거죠.

마지막으로 쿨러 사건을 맡은 기준 에이전트와 견줘 보겠습니다. 목표와 역할과 도구가 어떻게 다른지 적으면 돼요. 남는 결과물은 내가 만든 에이전트, 그리고 근거가 붙은 질문 한 건의 기록입니다.

첫 활동이 에이전트의 구성을 익히는 일이라면, 뒤의 세 활동은 그 구조를 서로 다른 업무에 적용하는 일입니다.

[SLIDE 58~59] 실습: 설비 이상의 원인 추론과 점검 순서 결정

두 번째 실습에서는 원인을 추론하고 점검 순서를 제안하는 에이전트를 만듭니다. 첫 실습의 포털 구성에서 한 걸음 더 들어가, 바이브 코딩으로 근거를 읽고 결과를 나누는 방식을 직접 구현할 거예요.

화면의 네 칸을 따라가 볼게요. 에이전트는 공용 온톨로지에서 설비, 센서, 고장 유형, SOP로 이어진 관계를 읽어 원인 후보를 얻습니다. 그다음 제공된 센서 입력을 증거 기준에 대어 보고요. 결과는 등록된 기준이 지지하는 원인, 아직 확인하지 못한 가설, 기준에 맞지 않는 후보로 나눠요. 그리고 마지막 칸, 다음에 점검할 걸 순서대로 적습니다.

성공 기준은 확인되지 않은 원인을 단정하지 않고 가설로 표시하는 거예요. 남는 결과물은 점검 계획 파일과, 그 계획에 쓴 근거입니다.

점검 계획에 원인 이름만 쓰는 것이 아니라, 왜 그 점검이 필요한지까지 남기는 활동입니다.

[SLIDE 60~61] 실습: 품질 불량 접수의 후속 업무 연결

세 번째 실습에서는 같은 원리를 품질 업무에 적용합니다. 앞의 원인 추론 활동이 센서와 증거 기준을 비교하는 일이라면, 이번 활동은 접수 메시지를 읽고 다음에 누가 무엇을 확인할지 정하는 일이에요. 제공된 접수 메시지와 분류 기준으로 품질 대응 카드를 만드는 작은 에이전트를 구현합니다.

에이전트가 낼 품질 대응 카드에는 정해진 항목이 있어요. 화면 앞의 세 카드가 그 항목입니다. 품질 유형, 추가로 확인할 항목, 그리고 담당 부서와 대응 초안이죠. 정해진 항목으로 내는 것, 앞에서 본 구조화 출력이에요.

꼭 넣어야 할 갈래도 하나 있습니다. 마지막 카드, 원인 미확정이에요. 원인이 정해지지 않은 접수는 원인을 단정하지 않고 조사 요청으로 넘겨야 해요. 성공 기준은 원인이 분명한 접수와 원인 미확정 접수가 서로 다른 카드로 나오는 거예요. 남는 결과물은 구조화된 품질 대응 카드입니다.

이 활동의 결과는 대응 카드와 초안입니다. 실제 부서에 메시지를 보내거나 업무를 접수시키는 것까지 완료했다고 다루지는 않습니다.

[SLIDE 62~63] 실습: 여러 센서의 이상 징후 탐색과 시각화

마지막 실습은 판단의 입력인 센서 값으로 돌아갑니다. 온도와 압력이 시간에 따라 어떻게 변했는지 도표로 보고, 더 살펴볼 구간을 찾는 활동이에요. 시간 축을 맞춰 비교하되 서로 다른 단위는 구분해서 표시해야 합니다. 온도와 압력의 숫자 크기만 놓고 어느 쪽이 더 심하다고 비교할 수는 없으니까요.

요약에는 이상 구간과 관련 태그, 조회한 기간을 근거와 함께 적습니다. 그리고 마지막 카드, 출력도 점검하겠습니다. 함께 변한 값을 고장 원인이라고 단정하지 않았는지가 성공 기준입니다. 확인된 원인과 가설을 가를 때 봤던 그 원칙이고, 품질 대응 활동에서 원인 미확정 접수를 조사 요청으로 구분할 때도 적용할 원칙이죠. 남는 결과물은 도표와, 이상 구간 · 관련 태그 · 조회 기간을 적은 요약입니다.

네 활동은 에이전트 구성에서 시작해 원인 추론, 품질 대응, 센서 탐색으로 이어집니다. 공통으로 확인할 것은 답을 그럴듯하게 냈느냐가 아니라, 맡긴 범위에서 근거를 확인하고 모르는 부분을 드러냈느냐입니다. 이제 첫 활동인 포털의 업무 에이전트 구성부터 시작하겠습니다.

[SLIDE 64] 쿨러 사건의 94초를 다시 읽기

네 실습에서 구성한 결과를 떠올리면서, 처음의 쿨러 사건을 다시 읽어 보겠습니다. 에이전트가 무엇을 확인했고 무엇을 사람에게 넘겼는지, 표의 다섯 장면으로 비교해 볼게요. 가운데 열의 물음에 먼저 답한 뒤 오른쪽 근거와 대조해 보세요.

첫 줄, 대안 평가가 네 후보의 점수를 냈어요. 계산한 건 누굴까요? 모델이 아니라 판단 도구입니다. 모델은 어느 도구에 뭘 넣을지 적었을 뿐이죠.

둘째 줄의 70 %와 약 46퍼센트, 둘 다 지금의 값일까요? 아니에요. 70은 지식 그래프에 적힌 기준이고, 약 46퍼센트가 그때 읽은 값입니다.

셋째 줄의 쿨러 핀 오염과 팬 구동 불량은 둘 다 원인 후보였어요. 이 모델의 증거 기준이 지지한 것은 쿨러 핀 오염이었습니다. 팬 진동이 기준보다 낮아서 팬 구동 불량은 지웠죠.

넷째 줄, 추천안 제출은 에이전트가 했고, 팬 속도 명령은 운전원이 승인한 뒤 시스템이 냈습니다. 마지막 줄의 조회 0행은 그 조건에 맞는 기록이 없다는 뜻이고, 조회 실패는 기록을 확인하지 못했다는 뜻이에요. 둘 다 재고 수량 0과는 구분해야 합니다.

계산의 주체, 지식과 값, 확인과 가설, 제안과 실행, 일치하는 기록 없음과 조회 실패. 이 다섯 짝을 가를 수 있으면, 에이전트가 낸 결과를 어디까지 믿어도 되는지 판단할 수 있습니다.

[SLIDE 65] 오늘 다룬 핵심 개념과 주요 용어 정리

사건 쪽은 이렇게 다시 읽어 봤고요. 다음은 이번 회차에 나온 용어를 정리해 보겠습니다. 표에 이번 회차에 다룬 아홉 개념의 이름과 뜻이 있어요.

LLM은 입력과 학습한 내용을 바탕으로 응답을 만드는 대규모 언어 모델이에요. RAG는 검색한 자료를 답 생성의 근거로 제공하는 방식이고요. 이 수업의 에이전트는 목표를 받고 도구의 결과에 따라 다음 행동을 정하되, 완료하거나 멈춤 조건에 이르면 끝냅니다. 도구는 실제 조회, 계산, 제출을 수행하는 기능입니다.

실행 고리는 요청 해석, 계획, 호출, 관찰을 되풀이하다가 종료하는 틀이에요. 신선도는 관측된 값이 판단 시점에서 얼마나 오래됐는지를 나타냅니다. 근거의 상태는 셋이었죠. 근거의 지지를 받음, 확인하지 못함, 지지 근거 없음.

구조화 출력은 정해진 항목과 형식을 갖춘 JSON 결과고요. 실행 기록은 도구 호출의 시각과 입력, 출력, 걸린 시간을 남긴 기록입니다.

[SLIDE 66] Review Quiz: 핵심 흐름 되짚어보기

이름과 뜻을 정리했으니, 그걸 판단에 쓸 수 있는지 퀴즈 세 문제로 확인해 보겠습니다. 용어를 외웠는지보다, 상황을 보고 에이전트가 무엇을 해야 하는지 고를 수 있는지가 중요해요. 문제마다 고를 시간을 드릴게요. 세 문항을 다 푼 뒤에 정답과 까닭을 같이 보겠습니다.

[SLIDE 67] Quiz 1. 처리 기록의 한 장면

첫 번째 문제입니다. 에이전트가 원인 진단을 불렀어요. 그리고 그 결과에 든 원인과 고장 유형의 식별자를 넣어서 대안 평가를 불렀습니다. 이 장면이 보여 주는 실행 고리의 단계는 뭘까요?

1번, 요청 해석.
2번, 결과 관찰에 이은 다음 호출.
3번, 재시도.
4번, 종료.

[SLIDE 68] Quiz 2. 읽지 못한 값과 묵은 값

두 번째 문제예요. 에이전트가 원인을 가리려고 합니다. 그런데 유온의 최신 값은 3분 전의 것이었고, 냉각 효율 조회는 실패했어요. 이때 에이전트의 올바른 동작은 무엇일까요?

1번, 3분 전의 유온과 판별 기준 70 %를 써서 쿨러 핀 오염으로 제출한다.
2번, 냉각 효율을 0 %로 적고 유온을 가장 많이 내리는 조치를 추천한다.
3번, 원인 후보에 올라 있는 쿨러 핀 오염을 확인된 원인으로 적는다.
4번, 원인을 적지 않고, 확인하지 못한 값과 그 이유를 담아 보류한다.

[SLIDE 69] Quiz 3. 허용되지 않은 도구

마지막 문제입니다. 에이전트가 일을 끝내려면 허용 목록에 없는 도구가 필요하다고 판단했어요. 유온은 계속 오르고 있고요. 이때 일어나야 하는 일은 무엇일까요?

1번, 허용된 도구를 여럿 조합해 같은 효과를 내는 길을 찾는다.
2번, 급한 상황이므로 한 번만 실행하고 그 사실을 기록에 남긴다.
3번, 실행하지 않고 사람에게 묻는다. 답을 받아도 실행 권한은 그대로다.
4번, 사람에게 물은 뒤 답을 받으면 그 도구를 직접 실행한다.

[SLIDE 70] 정답과 판단 기준

세 문제를 다 골랐으면, 정답을 확인해 볼게요. 고른 답과 그 까닭을 표와 함께 대조해 보세요. 첫 문항의 정답은 2번입니다. 서로 다른 두 도구를 이어서 불렀고, 앞 호출의 결과가 뒤 호출의 입력이 됐죠. 결과를 관찰한 게 다음 행동을 정한 장면이에요.

3번 재시도와 헷갈렸을 수 있어요. 재시도는 뜻대로 되지 않은 일을 방법을 바꿔서 다시 하는 겁니다. 이 장면에서는 앞 호출이 제대로 끝났고, 그 결과를 가지고 다른 도구를 불렀죠.

둘째 문항은 4번입니다. 묵은 값과 실패한 조회로는 근거가 확인되지 않아요. 1번은 3분 전의 값을 지금의 값처럼, 기준 70을 측정값처럼 썼습니다. 2번은 실패를 0으로 바꿨고, 3번은 후보를 확인된 원인으로 바꿨고요. 셋 다 결과의 모양은 그럴듯하지만, 근거가 없는 자리를 채운 거예요.

셋째 문항은 3번입니다. 허용 목록 밖의 도구는 실행되지 않고, 사람에게 묻는 질문으로 올라가요. 1번은 허용된 도구로 막힌 일을 돌아가는 거고, 2번은 급하다는 이유로 권한을 넘는 겁니다. 4번은 사람에게 물었다는 점에서 그럴듯하죠. 하지만 답변이 실행 권한을 바꾸지는 않아요.

처음에는 AI가 추천안을 냈다는 결과만 보였지만, 호출 기록을 펼치니 역할이 나뉘었죠. 모델은 다음 행동을 정하고, 도구는 실제 값을 읽고 계산하며, 사람은 근거와 권한의 경계를 확인합니다. 이 구분을 지켜야 다른 업무에 에이전트를 적용할 때도 무엇을 맡기고 무엇을 확인할지 정할 수 있습니다.

다음 회차에서는 에이전트가 읽은 업무 지침을, 다시 쓸 수 있는 능력으로 만드는 방법을 다룹니다. 그리고 에이전트가 여러 시스템의 도구를 어떻게 연결해 쓰는지, 그때 지켜야 할 안전 기준은 뭔지도 살펴볼 거예요. 이번 회차도 수고 많으셨습니다. 감사합니다.